

Chapter 11

MPEG-2 Systems

11.1. INTRODUCTION

The video and audio parts of the MPEG-2 standard define the bit-stream syntaxes and the signal processing required for the decoding of video and audio bit streams. Each of these bit streams, known as an *elementary stream*, is generated by an elementary stream encoder (either video or audio) and must be stored or transported across a communications system before being passed to an elementary stream decoder. The systems part of the MPEG-2 standard [1] defines a number of services that are required for the delivery of these elementary streams. These services are as follows:

- *Multiplexing.* Multiplexing is used to interleave two or more elementary streams into a single stream that can be carried on a communications channel. This is achieved using a packet-based multiplex. Multiplexing is discussed in Section 11.3.
- *Timing.* The major purpose of the timing function is to provide synchronization. First, it is necessary during decoding to synchronize the operation of a decoder to the timing of the encoder that generated the bit stream being decoded. Second, synchronization between compressed streams on decoding and playback must be provided. Without this type of synchronization, lip-sync cannot be provided. Timing is discussed in Section 11.4.
- *Buffer management.* A decoder stores a received bit stream until it is ready to decode it. Buffer management is required to ensure that the decoder's internal buffers do not overflow, resulting in the loss of parts of an elementary stream. Buffer management is discussed in Section 11.5.
- *Transmission of control data.* A variety of control data must be transmitted to carry configuration information about elementary streams (allowing them to be reassembled into audio-visual programs), the network carrying the multiplexed stream, and the conditional access system used. This data takes the form of a number of tables and is discussed in Section 11.6.

This chapter describes the structure of MPEG-2 systems and its use in the *digital video broadcast (DVB)* and *advanced television standards committee (ATSC)* suites

of standards. The aim is to provide the reader with an understanding of the operation of MPEG-2 systems but not to describe fully its syntax. A complete description of the syntax can be found in the MPEG-2 systems standard. A number of extensions to MPEG-2 systems have been defined by DVB and ATSC. These extensions both increase the functionality of the systems layer and ease the task of the decoder in some circumstances. They are discussed separately in Chapter 12.

11.2. SERVICE OVERVIEW

The structure of a simple MPEG-2 encoder [2] encoding a single program is shown in Figure 11.1. A video signal is converted into a video bit stream by the video encoder. An audio signal is converted into an audio bit stream by the audio encoder. These two elementary streams are conveyed to the multiplexer, which multiplexes them together and inserts control information to produce a single systems bit stream.

The systems bit stream, illustrated in Figure 11.2, contains interleaved video, audio, and control data. The interleaving is achieved by placing data from the elementary streams in packets. This allows much greater flexibility than would be possible with a periodic time-division multiplexing, particularly for streams that have vastly different data rates and streams whose data rate changes over time.

The various components making up an MPEG-2 systems bit stream are distinguished by the contents of the packet header, in which each component is allocated a unique value of the payload identifier (PID) field.

A television program encoded as an MPEG-2 bit stream contains, therefore, a number of multiplexed elementary streams, each of which contains data that can be decoded by either a video or audio decoder. The MPEG-2 decoder (Figure 11.3) [3] is responsible for decoding of the received systems bit stream and the reassembly of the video and audio components of the program. The systems decoder performs the first stage of decoding, extracting the elementary streams and passing them to the elementary stream decoders, that is, the video and audio decoders. The elementary stream decoders decode the elementary streams and perform the signal processing required to reconstruct the video and audio signals.

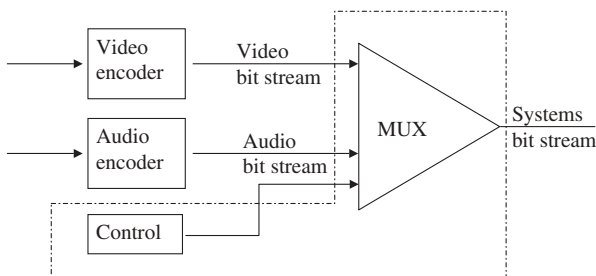


Figure 11.1 Example encoder.

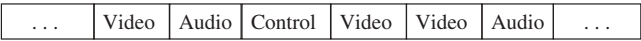


Figure 11.2 Example systems bit stream.

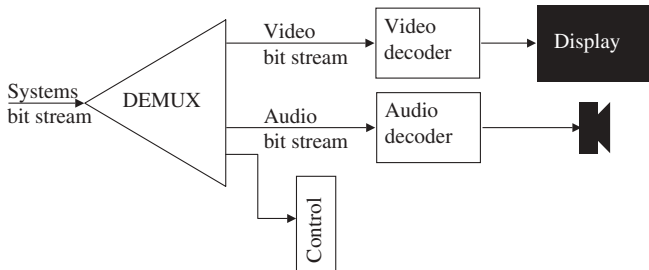


Figure 11.3 Example decoder.

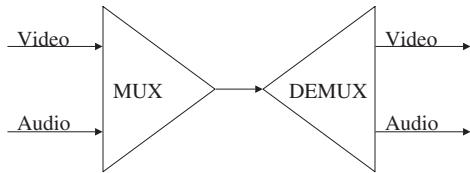


Figure 11.4 Single-program operation.

The systems encoding and decoding processes for a single program can be represented schematically as shown in Figure 11.4. The elementary stream encoders and decoders are not shown.

For a systems bit stream containing multiple programs, the systems encoder and decoder can be represented as shown in Figure 11.5. Elementary streams from the multiple programs are passed into the systems encoder. The resulting systems bit stream is then transmitted to the decoder over a single channel. The systems decoder extracts the elementary streams that make up the multiple programs, passing the streams corresponding to one program to the elementary stream decoders and discarding all other elementary streams.

The sharing of elementary streams between programs is also possible. In the example shown in Figure 11.6, a video stream is shared between two programs that have distinct audio streams. This approach might be used for the broadcast of a

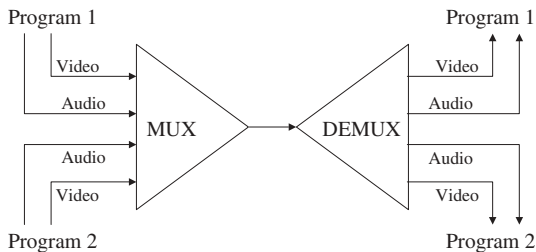


Figure 11.5 Multiple programs.

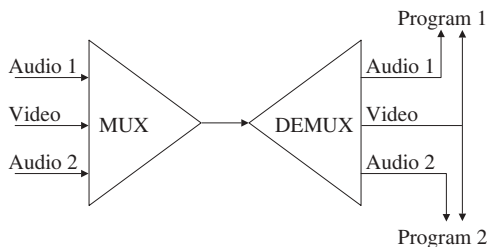


Figure 11.6 Shared elementary streams.

movie dubbed into a number of languages or for a sports broadcast to be viewed in several languages. In the latter case, the soundtracks may not only be in different languages but may also be generated independently, expressing national biases in the outcome of the event.

Although the primary types of data carried by the MPEG-2 systems layer are MPEG-2 video and MPEG-2 audio, it is often necessary to carry other types of data not standardized by MPEG-2. Such data, known as private data, may be used for the transfer of key management data for conditional access systems, the use of non-MPEG-2 audio standards (such as Dolby AC-3), or carrying information associated with a program such as subtitles. The combined use of video, audio, and private data elementary streams is illustrated in Figure 11.7. Both DVB and ATSC make extensive use of private data.

In order to allow synchronization between elementary streams, two types of timing information are carried in an MPEG-2 systems bit stream. Clock references are used to pass the current time to the decoder; time stamps provide information on the time at which specific actions are to occur.

The term *access unit* is used to describe the basic unit of data that is presented to the viewer by the decoder. For a video elementary stream, the access unit corresponds to a picture. For an audio elementary stream, the access unit corresponds to an audio frame.

The control information carried by the MPEG-2 systems layer provides information about timing, buffer management, and the structure of the bit stream, including the *program-specific information* (PSI) that allows a decoder to group elementary streams into programs and to decode one program from a systems bit stream containing several programs. The PSI is organized into a number of tables. Two of these tables effectively provide a table of contents for the bit stream: the

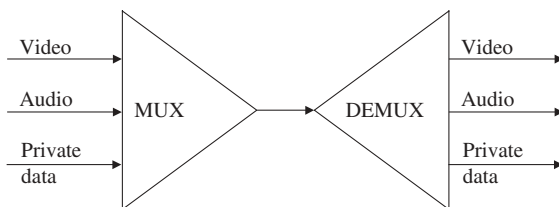


Figure 11.7 Use of private data.

program association table and the program map table. Each program has its own associated program map table, which contains a list of PIDs for the packets carrying the elementary streams making up the program. The program association table carries a list of PIDs for the packets carrying the program map tables of all program carried in the system bit stream, providing a single entry point for the system bit stream.

11.3. MULTIPLEXER STRUCTURE

The MPEG-2 systems multiplexer is a two-layer, packet-oriented multiplex. The upper sublayer, the *packetized elementary stream* (PES) sublayer, breaks the video and audio bit streams into packets and attaches a packet header that contains packetization, timing, and control information. The PES packets are passed to one of the two multiplexing sublayers. The *transport stream* (TS) is the more commonly used of the multiplexing sublayers and is designed to provide robust transmission on unreliable channels. The *program stream* is provided as an alternative to the transport stream and can be used to provide backward compatibility with MPEG-1. The DVB and ATSC standards for digital television use only the transport stream.

In this section, the properties of the MPEG-2 multiplexer are discussed.

11.3.1. PES Sublayer

The PES sublayer packetizes bit streams from elementary stream encoders. Each packet consists of a header, containing configuration information about the packet, timing information and information about the type of the payload, and a payload containing the elementary stream data.

The PES sublayer can be thought of as an adaptation layer between the continuous bit streams generated by the video and audio encoders and the multiplex of the transport stream or program stream. This adaptation layer captures a number of important types of information about the elementary stream including timing and control information. PES packets are passed to either the transport stream or program stream for multiplexing.

11.3.1.1. PES Packet Header

Each PES packet header begins with a 32-bit MPEG start code. The first 24 bits have value 0x000001. The remaining 8 bits are the **stream_id** of the data in the packet, which specifies the type of data carried by the PES packet. This **stream_id** is used extensively by both DVB and ATSC digital television systems.

The remainder of the PES packet header contains the following types of information: packetization information, which provides information about the structure of the packet; timing information, which is used by the decoder for synchronization; and control information, which provides other information about the elementary stream.

The major fields of the PES packet header are shown in Table 11.1. Fields carrying data that only describes the structure of the PES packet header have been omitted. All other fields are included. Even though the lengths of many of the fields in the PES packet header are not multiples of 8 bits, the syntax guarantees that the length of the header is always an integer number of bytes.

The length of a PES packet is specified in the 16-bit **PES_packet_length** field of the PES header. The maximum length that can be specified in this field is $2^{16}-1$ bits, which is the maximum length allowed for all PES packets except for those containing video elementary streams. In the case of video elementary streams, a value of 0 may be placed in this field, indicating that the length is unbounded. Where the PES length is unbounded, a decoder must determine the length from the transmitted bit stream. The reason that the unbounded length option is permitted for video elementary streams is that start-code emulation is prevented in these streams. (Chapter 5 provides an explanation of start-code emulation.) Because audio elementary streams

Table 11.1 Fields of PES packet header.

Field	Size (bits)	Use
packet_start_code_prefix	24	0x000001
stream_id	8	Identifies the type of the elementary stream, for example, video, audio, or private data
PES_packet_length	16	Number of bytes in the PES packet following this field
PES_scrambling_control	2	Indicates whether or not the payload of the PES packet is scrambled
PES_priority	1	“1” indicates higher priority than “0”
data_alignment	1	“1” indicates that the PES header is immediately followed by a video start code or audio sync word
Copyright	1	Indicates that the payload is protected by copyright
PTS	33	Presentation time stamp
DTS	33	Decoding time stamp
ESCR	42	Elementary stream clock reference
trick_mode_control	3	Signals the use of pause, fast forward, and fast reverse modes—primarily for video-on-demand applications
previous_PES_packet_CRC	16	Intended for network maintenance—only calculated on payload because header can be modified in transport
stuffing_byte	8	Up to 32 bytes of stuffing may be added to a PES header

do not use the same start-code structure, start-code emulation is therefore not prevented in these streams.

The use of scrambling on the body of the PES packet can be indicated using the **PES_scrambling_control** field. The binary value 00 indicates no scrambling; the remaining three values indicate that scrambling is used. Differences in the meanings of these three values are not defined by MPEG-2 but are left to vendors of conditional access systems.

The primary type of timing information carried by the PES packet header is time stamps. The structure of these fields is discussed in Section 11.4. Two types of time stamps may be used: PTS and DTS. The PTS tells the decoder the time at which decoded information should be presented to the viewer and may be used with video, audio, and private data. The decoding time stamp is for use where a picture must be decoded before it is displayed and tells the decoder when decoding should begin. (More information on clock references is provided in Section 11.4.2.) Clock references can also be carried in the PES header using the elementary stream clock reference (ESCR). In principle, this means that each elementary stream can be synchronized to an independent clock. In digital television, all streams making up a program are usually required to be synchronized to a single clock, and this is not used.

The **stuffing_byte** may be used to pad a header. One use of this field is to allow the length of the PES header to remain unchanged when a field is deleted during transmission. This might occur when a time stamp is deleted during transcoding.

Alignment between the boundaries of PES packets and the video bit stream offers a number of advantages, especially for error resilience. Byte alignment is always preserved, because a PES packet always contains a whole number of bytes of the video bit stream. Byte alignment of start codes in the video bit stream is achieved by inserting 0 bits at the end of a video bit stream until its length in bits is a multiple of 8. Setting the **data_alignment** field of the PES header to 1 indicates that the PES header is immediately followed by a video start code or audio sync word. For video, this means that the PES packet is aligned with the slice layer, picture layer, GOP layer, or sequence layer. Further information on alignment may be carried in the PSI referring to this stream as discussed in Section 11.6.2.2.

11.3.1.2. Overheads Due to the PES Layer

The basic length of the PES header used with video and audio elementary streams is 8 bytes. A major component of this first part of the header is a group of flags that indicate the presence or absence of other parts of the PES header. Where a time stamp is present in the PES header, an additional overhead of 5 bytes is added to the basic overhead. Time stamps are required no more often than once per picture and are often required much less often. Their overhead amounts to much less than 0.1% of the total rate. Where the ESCR is present, a further 6 bytes of overhead is added. A further 2 bytes is added if the optional CRC is transmitted, which does not usually occur in normal transmission. Further variable overheads are added by the use of trick modes, or inclusion of private data or stuffing bytes in the header.

The average overhead of the PES layer is therefore approximately 8 bytes per PES packet, with the additional data added by the use of optional fields not contributing significantly. The total overhead (O) therefore depends on the number of PES packets per second (P) and the data rate in bytes per second (R):

$$O = 8 \frac{P}{R}$$

EXAMPLE 11.1—PES Layer Overheads

Calculate the percentage overhead for a 4-Mbit/s, Rec.601-resolution video sequence (25 pictures per second, with each picture containing 576 lines) with one slice per row of macroblocks and one PES packet per slice. Repeat the calculation for HDTV (1088 lines per picture, with a frame rate of 25 frames per second) at 20Mbit/s.

In 576 lines, there are 36 rows of macroblocks. With one slice per row of macroblocks and one PES packet per slice, we have $25 \times 36 = 900$ PES packets per second or 7200 byte/s. For a video elementary stream at Rec.601 resolution and 4Mbit/s (500 Kbyte/s) with one PES packet per slice, the overhead is therefore 1.4%.

In 1088 lines, we have 68 rows of macroblocks. With one slice per row of macroblocks and one PES packet per slice, we have $25 \times 68 = 1700$ PES packets per second or 13.6 Kbyte/s. For HDTV at 20Mbit/s (2.5 Mbyte/s) with one PES packet per slice, the overhead is approximately 0.54%. The use of larger PES packets reduces these overheads to even lower levels. ■

11.3.2. Transport Stream Sublayer

The purpose of the transport stream sublayer is to provide a robust multiplex for PES packets from a number of elementary streams. All data is carried in packets with length 188 bytes. This packet length is chosen to allow an MPEG-2 transport stream packet to fit into four asynchronous transfer mode (ATM) cells, allowing for the overhead of the ATM adaptation layer. The first four bytes of each packet are the packet header. The remaining bytes may optionally contain an adaptation field, payload bytes, or an adaptation field followed by payload bytes.

Each section of each PSI table is carried as the payload of one or more transport stream packets. Multiple sections with a common PID may be carried in a single transport stream packet.

11.3.2.1. Transport Stream Packet Header

The allocation of the contents of the transport stream packet header is shown in Table 11.2. The first byte of the transport stream packet header is the **sync_byte**. This field always has the value 0x47 and is used for synchronization. When a decoder begins the decoding of a new transport stream, it first searches the stream for the value 0x47. Because there is no protection against emulation of the **sync_byte** (unlike the start codes used in MPEG-2 video), the decoder must ensure that

Table 11.2 Transport stream packet header. Fields are transmitted in the order indicated. Each field is transmitted with its most significant bit first.

Field	Size (bits)	Use
sync_byte	8	Always 0100 0111 (0x47)—used for Synchronization.
transport_error_indicator	1	Set if network's transport layer detects an error.
payload_unit_start_indicator	1	Packet begins with new PES packet or PSI section.
transport_priority	1	"1" indicates higher priority than "0."
PID	13	Payload ID; indicates number and type of elementary stream.
transport_scrambling_control	2	Indicates use of encryption.
adaptation_field_control	2	Indicates presence or absence of adaptation field and payload.
continuity_counter	4	Increases with each transport stream packet with same PID. Can be used to detect lost packets.

a number of correct **sync_byte** fields are received spaced exactly 188 bytes apart. The **transport_error_indicator** is usually set to 0, but may be set to 1 where the transmission network's transport layer detects an error that is passed uncorrected to the MPEG-2 decoder.

The **payload_unit_start_indicator** field is used to indicate whether the transport stream packet contains the first byte of a new PES packet or a new section of a PSI table. If the transport stream carries PES data, setting the **payload_unit_start_indicator** to 1 indicates that the payload of the transport stream packet begins with a new PES packet, that is, the PES start code consisting of 0x000001 followed by the 8-bit **stream_id**; a value of 0 indicates that no PES packet starts in this transport stream packet. For transport stream packets carrying PSI data, setting the **payload_unit_start_indicator** to 1 means that the first byte of the payload of the transport stream packet points to the start of the first new PSI section to start in this transport stream packet. The possible uses of **payload_unit_start_indicator** are illustrated in Figure 11.8.

The **transport_priority** field may be used by an encoder to indicate to the transport layer of the transmission network that some transport stream packets have higher priority than others. This may be useful in distribution systems, such as those based on ATM switching, that are able to provide higher levels of protection against error and loss to some packets. The **PID** identifies the elementary stream with which the transport stream packet is associated. The 13 bits assigned to this field permit up to 8192 different streams to be uniquely identified. Some values of this field are preassigned, such as 0x0000 being used for the program map table. A **PID** value of 0x47 should not be used, as this may be confused by a decoder with the **sync_byte**. The **transport_scrambling_**

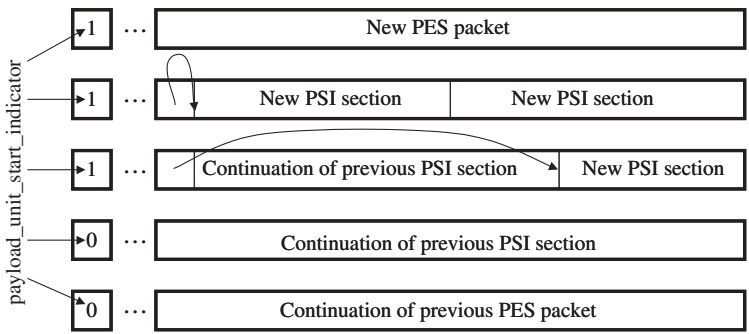


Figure 11.8 Uses of `payload_unit_start_indicator` in the MPEG-2 transport stream packet header.

control field is used to indicate if the contents of the transport stream packet are scrambled for conditional access. A value of 0 indicates no scrambling. All other values indicate the use of conditional access. Definition of any difference in meaning between the available nonzero values is left to the implementor. The **adaptation_field_control** indicates whether or not the adaptation field (described below) and payload are present in the transport stream packet. Two bits are required to signal one of the three possible packet configurations: adaptation field only, payload only, or both adaptation field and payload. The **continuity_counter** is increased by one for each successive packet having the same value of **PID**.

11.3.2.2. Adaptation Field

Table 11.3 shows a summary of the contents of the adaptation field, listing fields whose value impacts on the remainder of the transport stream but omitting flags and field lengths that affect only the contents of the adaptation field.

The purposes of the adaptation field are to carry the clock references used by the decoder to recover information about the timing of the stream (described in Section 11.4) and to provide information to assist buffer management, including notification of discontinuities in the clock reference, splicing points, and the latest valid arrival time for a transport stream packet. Without these clock references, a decoder would not be able to reliably decode a received bit stream and to regenerate the video frame and audio sample clocks.

Splicing points provide support for the editing of MPEG-2 systems bit streams. Editing without the use of splicing points requires that the system time clock run continuously, that is, without break. The major impact of this is that when two MPEG-2 bit streams are spliced together, the timing of one must be altered so that the system time clock is continuous across the point where they are joined. This usually requires that the transport stream of at least one of the sequences is recoded, although it may be possible to avoid recoding of video and audio material. The VBV requirements (see Chapter 6) of video, however, must also be met. Meeting this requirement may sometimes require recoding of video.

Table 11.3 Summary of adaptation field contents.

Field	Size (bits)	Use
discontinuity_indicator	1	Indicates discontinuity in system time clock—the next program clock reference in a transport stream packet with same PID begins a new system time clock.
random_access_indicator	1	Current transport stream packet contains information to aid random access—a video sequence header for a video elementary stream or the start of an audio frame for an audio elementary stream.
elementary_stream_priority_indicator	1	Indicates priority of transport stream packet within elementary stream.
program_clock_reference	42	PCR—this clock reference is used by the decoder to reconstruct the STC.
original_program_clock_reference	42	Original PCR—used to transmit the original value of the PCR when it has been modified during remultiplexing or transcoding.
splice_countdown	8	Number of transport stream packets of same PID until a splicing point.
transport_private_data_length	8	Number of bytes of private data contained in this transport stream packet.
ltw_offset	15	Legal time window in which transport stream packet should arrive to prevent illegal buffer conditions.
piecewise_rate	22	Notional rate used to define end of legal time window.
splice_type	4	Assists the decoder to prevent buffer overflow at the next splice.
DTS_next_AU	33	Decoding time stamp for the next access unit, which begins in the transport stream packet after splice_countdown reaches 0.

The purpose of splicing points in MPEG-2 systems bit streams is to avoid the need for recoding of either systems bit streams or elementary streams during editing. Where a splice occurs in an MPEG-2 systems bit stream, the **discontinuity_indicator** field of the transport stream packet after the splice is set to 1. This means that the **continuity_counter** field of the header of the first transport stream after the splice may have an arbitrary value. The **discontinuity_indicator** field of the transport stream packet after the splice carrying the program clock reference (PCR) is also set to 1. This means that the decoder must resynchronize to a new system time clock. The **splice_countdown** field of the adaptation field of transport stream packets preceding the splice is used to inform the decoder of the precise location of the splice. The **splice_type** field in

the adaptation field of transport stream packets leading up to the splice is used to provide information to the decoder on the peak data rates that can be expected as a result of the splice, assisting the decoder with its buffer management.

Stuffing bytes, whose value is 0xFF, are available in the adaptation field. These are used to fill up transport stream packets that are only partially filled by the payload and adaptation fields. This might happen in the last transport stream packet of a PES packet.

Null transport stream packets, whose PID is always 0x1FFF, are allowed. These transport stream packets can be inserted and deleted when transport streams are multiplexed to provide fine adjustment of bit rate. They can also be inserted in place of transport stream packets from a program that is deleted from a transport stream. The payload bytes of a null transport stream packet may take any value.

Private data may also be carried in the adaptation field. This private data may be used to extend the functionality of the adaptation field but is obviously available only to proprietary decoders.

An adaptation field containing a single clock reference is 8 bytes in length.

11.3.2.3. Transport Stream Payload

The payload of a transport stream packet consists of elementary stream data or PSI data. Each transport stream packet may carry data from only one elementary stream. Alternatively, a transport stream packet may carry data from one or more PSI sections. Multiple PSI sections may be carried in one transport stream payload, subject to the constraint that these PSI sections have the same associated PID. This often prevents carrying sections from multiple tables in a single transport stream packet.

11.3.2.4. Alignment between Transport Stream and PES

Byte alignment of the PES packet is always preserved by the transport stream. PES packets may only begin at the beginning of a transport stream packet. The last transport stream packet carrying a PES packet or PSI section may be only partially full; the remaining bytes of this transport stream packet have the value 0xFF.

11.3.2.5. Overhead Due to Transport Stream

The overhead due to the transport stream packet header (O_1) is 4 bytes per packet (total length 188 bytes), giving $O_1 = 2.1\%$, which is mostly associated with synchronization. Additional overheads come from the last transport stream packet carrying a PES packet being only partially full and the transmission of the adaptation field.

The amount of overhead due to partially full packets depends on the alignment between elementary stream (video, audio, or private) data and the PES layer and the proportion of the last transport stream packet that is empty. We assume that this

last packet is on average half full. Then the overhead due to partially full transport stream packets (O_2) depends on the number of transport stream packets per second (T) and the number of PES packets per second (P):

$$O_2 = \frac{0.5P}{T}$$

The overhead due to the adaptation field depends on how often this field is transmitted. In a transport stream, it is necessary to transmit the clock at least once every 100 ms, that is, 10 times per second (see Section 11.4). The clock reference is transmitted more regularly where there are discontinuities in the clock reference due to, for example, splicing. Based on the minimum 8 bytes required for an adaptation field containing a single clock reference, an overhead of 80 bytes per second is required for each program. This is insignificant for digital television applications.

The significant sources of overhead are therefore due to the transport stream packet header and the use of partially filled transport stream packets due to alignment between the PES packets and elementary streams. The total overhead O is therefore the sum of O_1 and O_2 :

$$O = \frac{4}{188} + \frac{0.5P}{T}$$

EXAMPLE 11.2—Overhead Due to Transport Stream

Calculate the overheads due to the transport stream for Rec.601 video (25 pictures per second, with each picture containing 576 lines) at 4 Mbit/s and HDTV (1088 lines per picture, with a frame rate of 25 frames per second) at 20 Mbit/s, assuming alignment between the PES and video layers at either the slice or picture layer. Assume that each slice contains one complete row of macroblocks.

For all cases, the overhead due to the transport stream packet header is approximately 2.1%.

For Rec.601 video, there are $576/16 = 36$ rows of macroblocks per picture, or 900 rows of macroblocks per second, corresponding to 900 PES packets per second if the alignment between PES and video layers is at the slice layer. For each of these PES packets, there is an average of one transport stream packet that is approximately half full. At 4 Mbps (500 kbyte/s), there are $500,000/184 = 2717$ transport stream packets per second. The overhead due to partially full transport stream packets is therefore approximately 16.6%, giving a total overhead of approximately 19%.

If the alignment between PES and video layers is at the picture layer, there are 25 PES packets per second. The overhead for this case is therefore approximately 0.5%, giving a total overhead of approximately 2.6%.

For HDTV, with alignment between video and PES at the slice layer, there are $25 \times 68 = 1700$ PES packets per second. On average, each of these leads to one transport stream packet that is half full. For HDTV at 20 Mbps (13,587 transport stream packets per second), the overhead due to partially full packets is approximately 6.3%. With alignment between video and PES at the picture layer, the overhead due to partially full transport stream

Table 11.4 Summary of transport stream overheads for video elementary streams in digital television applications.

	Rec.601 at 4Mbps	HDTV at 20Mbps
One slice per PES packet	19%	8%
One picture per PES packet	2.6%	2.2%

packets is approximately 0.1%. (This latter case would almost certainly require the use of the unbounded option on the **PES_packet_length** field.)

These overheads are summarized in Table 11.4. ■

11.3.3. Program Stream Sublayer

The program stream is provided for backward compatibility with MPEG-1 systems. It supports variable length packets, formed by grouping PES packets into *packs*. Each pack consists of a pack header possibly containing an optional system header, and zero or more PES packets. Although the program stream is used for DVDs, it is not used in either DVB or ATSC and is not discussed further here.

11.4. TIMING

The MPEG-2 timing functions are required to provide synchronization of the decoder to the encoder and between the elementary streams making up a program. Without synchronization of the decoder to the encoder, the decoder's video frame clock may run slower than the encoder's. Even if the difference in clock speed is very small, video frames will eventually have to be dropped when the internal buffers of the decoder overflow. On the contrary, if the decoder's video frame clock runs faster than the encoder's, extra video frames will have to be inserted by playing back some frames twice. In addition, if synchronization between elementary streams is not possible, the decoder is unable to preserve features such as lip synchronization in corresponding audio and video streams.

Synchronization of the decoder to the encoder requires that the two devices have the same concept of time. Synchronization between elementary streams requires that the encoder can pass information to the decoder on the times at which events associated with an elementary stream should occur. The common concept of time used by the encoder and decoder is the *system time clock* (STC). Inclusion of regular *clock references* in an MPEG-2 systems bit stream allows the transfer of the STC from encoder to decoder, allowing synchronization of decoder to encoder. The use of *time stamps* associated with events in individual elementary streams permits synchronization between these elementary streams. The concepts of the STC, clock references, and time stamps (illustrated in Fig. 11.9) are discussed in the following sections.

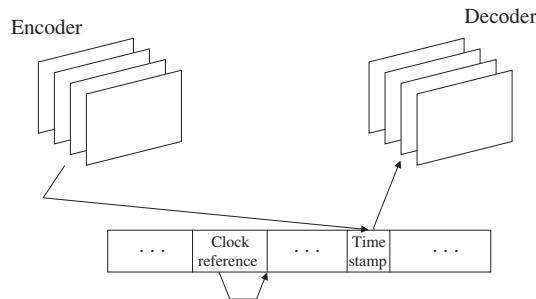


Figure 11.9 Time stamps and clock references.

11.4.1. System Time Clock

All timing in an MPEG-2 encoder or decoder is defined with respect to the STC. This 27-MHz clock is notionally generated by all encoders and must be reconstructed by decoders from clock references transmitted in the bit stream. A decoder can use the STC to manage its internal buffers, to generate the video frame and audio sample clocks, and to identify the precise time at which presentation units should be displayed. The relationship between the 27-MHz STC and the notional frequencies of the various video and audio clocks is shown in Table 11.5.

Each program in an MPEG-2 transport stream may have its own STC. There is no requirement for any timing relationship between these different STCs. Even within one program, discontinuities in the STC are permitted. These are commonly introduced at splicing points, where material from different sources is joined.

11.4.2. Clock References and Reconstruction of the STC

Clock references transmitted in an MPEG-2 systems bit stream are used by a decoder to reconstruct the STC. In simple terms, this means that clock references are used by the decoder to find out what the current time is.

Table 11.5 Divisors required to generate notional video and audio clocks from 27-MHz STC.

Clock	Divisor
25-Hz video frame	1,080,000
50-Hz video field	540,000
30-Hz video frame	900,000
60-Hz video field	450,000
44.1-kHz audio	30,000/49
48-kHz audio	1,125/2

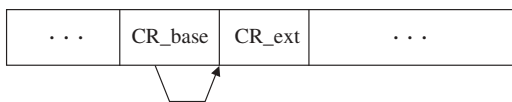


Figure 11.10 Timing of clock reference.

A clock reference is transmitted in the bit stream in two parts whose length totals 42 bits. The first is the **CR_base** (33 bits) and the second is the **CR_ext** (9 bits). The **CR_base** field specifies the time with respect to a 90-kHz clock derived by dividing the STC by 300. The **CR_ext** field specifies the remainder from this division. The full clock reference is used to reconstruct the STC. Time stamps are specified with respect to the 90-kHz clock of the **CR_base**.

The value of the clock reference is specified with respect to the location of the last bit of **CR_base** as illustrated in Figure 11.10.

The decoder uses the received clock references to regenerate the STC. This may be achieved by the use of a phase-locked loop (Fig. 11.11), which is used to adjust the operating frequency of a 27-MHz oscillator so as to keep it synchronized to a corresponding oscillator in the encoder. The phase-locked loop works by comparing received clock references to its internally generated STC. The differences are passed into a low-pass filter, whose output is used to speed up or slow down the oscillator from which the internal STC is derived. The use of a well-designed low-pass filter prevents jumps in the value of the STC and short-term, large variations in the operating frequency of the oscillator.

Simpler decoders may use a free-running oscillator and simply update the current value of the STC every time a clock reference is received, reducing complexity by removing the need for a phase-locked loop. Such decoders are likely to be particularly susceptible to delay jitter in the distribution system, which can cause larger jumps in the value of the STC.

Where STC discontinuities are signaled in the systems stream, the MPEG-2 decoder discards the memory in its clock recovery circuit, allowing the new STC to be generated. In the case of the structure shown in Figure 11.11, this requires setting to 0 all the values stored inside the low-pass filter. In digital television applications, the interval between clock references transmitted by an encoder must never exceed 100 ms. STC discontinuities may cause additional clock references to be transmitted.

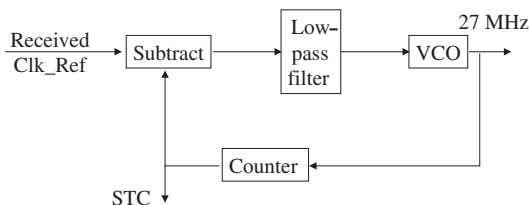


Figure 11.11 Outline structure of clock recovery circuit.

EXAMPLE 11.3—Clock Reference

How is the STC value 4,505,342 represented?

For an STC value of 4,505,342, the value of **CR_base** is the integer part of $4,505,342/300 = 15,017$ (represented in hexadecimal as 0x000003AA9). The value of **CR_ext** is the remainder from the division $4,505,342/300$, which is 242 (0xF2 in hexadecimal). ■

11.4.3. Time Stamps

Time stamps tell a decoder when an event associated with an elementary stream should occur. There are two types of time stamp: the *presentation time stamp* (PTS) and the *decoding time stamp* (DTS). The PTS is used to determine the display time of, for example, a video picture. This allows synchronization between elementary streams and generation of accurate video frame and audio sample clocks. The DTS may be used by a decoder to determine when it is necessary to start decoding the data for an access unit. The specification of DTS is based on the *system target decoder* (STD), which is a notional decoder architecture specified by MPEG-2. For decoders with different architectures, the DTS can be translated to a value appropriate to that architecture. The DTS is used only where the decoding time differs from the presentation time. One example is in a video sequence incorporating B-pictures, where I and P-pictures immediately following a B-picture (in presentation order) are transmitted before the B-picture.

Each time stamp is encoded as a 33-bit field. These 33 bits specify the time at which an event (presentation or decoding) associated with an elementary stream occur. The value of the time stamp provides the time with respect to the 90-kHz clock derived by dividing the STC by 300 and encoded as the base part of the clock reference.

11.5. BUFFER MANAGEMENT

The basic structure of an end-to-end MPEG-2 system is shown in Figure 11.12. Video and audio arrive at the left side of their respective elementary stream encoder, pass through the system and emerge at the right side with all timing relationships preserved. For the usual case of a constant video picture rate and a constant audio sample rate, this means that the video picture rates at either end of the system are the same, as are the audio sample rates. Any timing relationship between video and audio is also preserved.

In a simple system, the timing relationships may be preserved at all points through the chain from encoder to decoder. Because this would imply, for example, that all macroblocks of video were coded with the same number of bits, this situation never arises in practice. The usual situation is that the output of the video encoder has constant picture rate but variable bit rate. The buffer converts this output into a constant rate bit stream. Because the number of bits used to code each picture is not constant, pictures occupy different amounts of time in the output bit stream of the buffer.

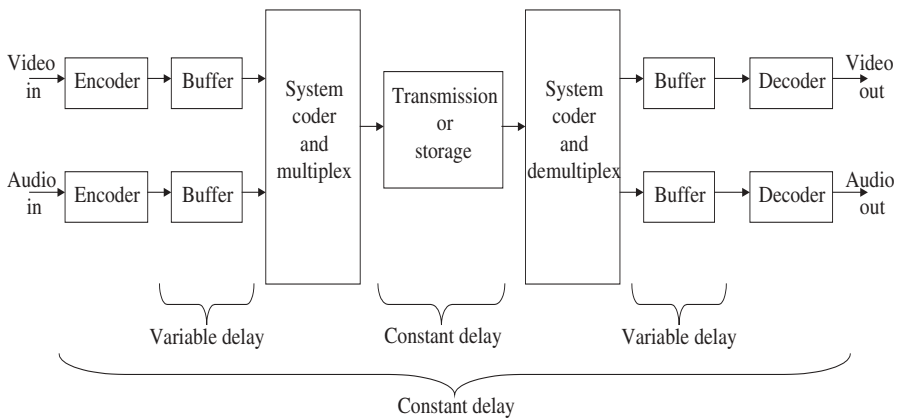


Figure 11.12 Basic structure of end-to-end MPEG-2 system.

A smaller amount of variability in the delay is introduced by multiplexing services at different rates. If, for example, there are 10 packets of video generated for each packet of audio, each of the 10 consecutive video packets experiences a different delay in multiplexing, which depends on how close it lies to an audio packet. If there were two audio packets for every nine video packets, then the audio packets experience variable multiplexing delay as well.

The bit stream that travels through the channel can be thought of as being constant bit rate but variable sample rate (i.e., variable picture rate for video). At the decoder, the buffer converts this signal back into a constant sample rate, variable bit-rate stream that is passed to the elementary stream decoder.

Practical decoders have finite-length buffers. Providing a high-quality service requires that the bit stream is such that these buffers do not overflow. This guarantee is provided by placing constraints on the minimum-length buffers that a decoder provides and on the amount of variability in sample rate that can be introduced into a bit stream by the encoder buffering.

An important consequence of the requirement to preserve timing relationships through encoding, multiplexing, demultiplexing, and decoding is that the end-to-end delay is constant. This means that an audio sample and a video picture arriving at the encoder at the same time will also be the output from the decoder at the same time.

The STD is defined by MPEG-2 as a reference structure against which parameters such as timing and buffer sizes can be defined. The STD defines the interconnections, transfer rates, and buffer sizes for a notional decoder, in which it is assumed that the elementary stream decoders operate instantaneously. There is no requirement, however, that a manufacturer implement this architecture in a real decoder. The purpose of the STD is to provide a framework allowing manufacturers to build decoders that are capable of decoding all compliant bit streams without loss of data due to buffer overflows. In other words, specifying buffer sizes with respect to the STD provides enough information so that a manufacturer can determine the buffer sizes required for a different decoder architecture.

11.6. PROGRAM-SPECIFIC INFORMATION

The MPEG-2 PSI provides information to the decoder on the structure of the MPEG-2 systems bit stream. This information includes the structure of the transport stream itself, the grouping of elementary streams to form programs, the types of these elementary streams (i.e., video, audio, or private data), the conditional access system (if any) used, and details of the distribution network. This information is contained in four tables: the *program association table* (PAT), the *program map table* (PMT), the *conditional access table* (CAT), and the *network information table* (NIT).

The program map table and conditional access table provide information about the *programs* carried in the transport stream. An MPEG-2 program can be thought of as comprising a set of elementary streams that are decoded and displayed simultaneously. Programs may share elementary streams. Elementary streams can exist for which there is no PSI information, which are therefore not part of any program and would not normally be decoded or displayed. PSI information may also be provided for programs whose elementary streams are not contained in the transport stream. The program map table and conditional access table carry most of their data in descriptors. Depending on where they are placed, a descriptor may refer to a single elementary stream, to all the elementary streams making up a program, or to the conditional access system.

The specific purposes of the tables are as follows:

- *Program association table.* The program association table is used to inform the decoder of the PIDs in which the program map table is carried for each program as well as the PID used for the network information table (if present).
- *Program map table.* Each program has its own program map table section, which specifies the PIDs of the elementary streams that make up the program and the type of each of these streams. Descriptors in the program map table may be associated with the whole program or with one elementary stream.
- *Conditional access table.* Where encryption is used to provide conditional access, the conditional access table is used to inform the decoder of the type of encryption used. A descriptor placed in the conditional access table provides system-wide information on conditional access.
- *Network information table.* A decoder requires a variety of information about the transmission network in order to receive and decode the transport streams carried by it. This type of information may be carried by the network information table.

11.6.1. MPEG-2 Descriptors

Most of the data in the PSI is carried in the descriptors. This section describes the generic syntax of MPEG-2 descriptors, before describing the structure of each individual descriptor.

Table 11.6 Generic syntax of descriptors used in MPEG-2 PSI tables.

Syntax	Number of bits	Mnemonic
<i>type_descriptor()</i> { descriptor_tag descriptor_length ... }	8 8	uimsbf uimsbf

11.6.1.1. Generic Descriptor Syntax

Because of the great variability in the types of information specified in the program map table and conditional access table, specific fields are not defined for most types of data carried in these tables. The basic idea of a descriptor is that it carries information on both the type of the data and its value. This is in contrast to most data in MPEG-2, where the type is not explicitly transmitted. The generic syntax of a descriptor contained in an MPEG-2 PSI table is shown in Table 11.6. (This pseudo-C syntax is explained in Chapter 5.)

The **descriptor_tag** field uniquely identifies the descriptor. For example, the value 0x02 identifies the descriptor as a `video_stream_descriptor`. The meanings of the possible values of the **descriptor_tag** field are shown in Table 11.7. The descriptor type is independent of the table in which the descriptor is transmitted.

Table 11.7 Descriptor names and purposes associated with values of the `descriptor_tag`.

<code>descriptor_tag</code>	Descriptor	Purpose
0	Reserved.	Not currently used.
1	Reserved.	Not currently used.
2	Video stream descriptor.	Provides basic information about a video elementary stream, such as profile and level, frame rate, and chroma format.
3	Audio stream descriptor.	Provides basic information about an audio elementary stream, including the type (e.g., layer number) of the audio stream.
4	Hierarchy descriptor.	Identifies program elements containing components of hierarchically coded audio and video, including spatial, temporal, and SNR scalability.
5	Registration descriptor.	Identifies formats for private data.
6	Data stream alignment descriptor.	Used to indicate the type of alignment for an elementary stream with data alignment == 1 in the PES header. For example, in video, alignment may be at the slice, picture, GOP, or sequence layer.

Table 11.7 (Continued)

descriptor_tag	Descriptor	Purpose
7	Target background grid descriptor.	Specifies the size of the display area where a decoded video stream is not to occupy the whole display area.
8	Video window descriptor.	Specifies the offset from the top left corner of the display area where a decoded video stream is not to occupy the whole display area.
9	CA descriptor.	Used in the PMT to indicate that conditional access is used to scramble a program. The type of CA used is specified. It may be used to carry (as private data) system-wide information (in the CAT) or elementary-stream-specific information (in the PMT)
10	ISO 639 language descriptor.	Specifies the language of the associated program (e.g., French) and the audio type (e.g., hearing impaired or visual impaired commentary).
11	System clock descriptor.	Specifies the source and accuracy of the clock used to generate clock references.
12	Multiplex buffer utilization descriptor.	Provides bounds on the occupancy of system target decoder multiplex buffers. Primarily intended for use in remultiplexing.
13	Copyright descriptor.	Identifies the owner of copyright on a program element and carries private data specific to the copyright owner.
14	Maximum bit-rate descriptor.	Indicates the maximum bit rate in a program.
15	Private data indicator descriptor.	Contains 32 bits of private data.
16	Smoothing buffer descriptor.	Specifies the size and leak rate of the smoothing buffer associated with a program element. This descriptor can be used by a decoder to assist with buffer management.
17	STD descriptor.	Used in buffer management for video streams.
18	IBP descriptor.	Contains information on the sequence of picture types in a video sequence.
19–63	Reserved.	Reserved for future ISO use.
64–255	User private.	Available for private definition.

At least in principle, this means that any descriptor can be transmitted in either the conditional access table or program map table. The **descriptor_length** field specifies the number of bytes of data contained in the descriptor following the **descriptor_length** field. The use of the **descriptor_length** field allows a decoder that is unable to decode a particular descriptor to skip over its data and begin decoding the data that immediately follows. The remaining bits making up a

descriptor carry the value of the descriptor. The syntax of these bits is defined separately for each descriptor.

11.6.1.2. MPEG-2 Audio Stream Descriptor

The MPEG-2 audio stream descriptor is used to provide information on an MPEG-2 audio elementary stream as shown in Table 11.8. The **free_format_flag** is set to 1 where one or more audio frames may have a value of 0 in the **bitrate_index** field in the audio elementary stream. The **ID** and **layer** fields have the same meanings as the corresponding fields in the associated audio elementary stream (Chapter 9). The **variable_rate_audio_indicator** is set where the bit rate may vary between consecutive audio frames.

A simplified representation of the structure of the MPEG-2 audio stream descriptor is shown in Figure 11.13, leaving out fields that are only used in the decoding of the descriptor. In this representation, an “*” next to a field represents a field that is optional or may be repeated, whereas a brace indicates that a group of fields is optional or may be repeated. This simplified representation is used throughout the remainder of this chapter and also in Chapter 12. Readers requiring more detailed information on the syntax of the various descriptors are referred to the MPEG-2 systems standard [1].

Table 11.8 Syntax of the MPEG-2 audio stream descriptor.

Pseudo-C syntax	Number of bits	Data type	Description
audio_stream_descriptor() {			Function header—does not contribute bits to the stream.
descriptor_tag	8	uimsbf	Standard descriptor header—see Table 11.6
descriptor_length	8	uimsbf	
free_format_flag	1	bslbf	Data from MPEG-2 audio header describing the audio elementary stream—specific to the audio stream descriptor.
ID	1	bslbf	
Layer	2	bslbf	
variable_rate_audio_indicator	1	bslbf	
Reserved	3	bslbf	Pad to byte boundary.
}			Function close—contributes no bits to the stream

audio_stream_descriptor	
free_format_flag	flag
ID	flag
layer	{ 1, 2, 3 }
variable_rate_audio_indicator	flag

Figure 11.13 MPEG-2 audio stream descriptor.

EXAMPLE 11.4—Audio Stream Descriptor

*Using the simplified representation, what are the contents of the audio stream descriptor describing a constant bit-rate audio stream with the **bitrate_index** fields taking values other than 0000 and **ID** field set to 1 and **Layer** 2?*

The audio stream descriptor carrying this information is shown in Figure 11.14.

audio_stream_descriptor	
free_format_flag	0
ID	1
layer	2
variable_rate_audio_indicator	0

Figure 11.14 Example MPEG-2 audio stream descriptor. ■

11.6.1.3. MPEG-2 Video Stream Descriptor

The purpose of the video stream descriptor (Fig. 11.15) is to provide information about one video elementary stream. This descriptor is usually carried in the program map table.

The video frame clock associated with a video elementary stream may be fixed or variable rate, as indicated by the **multiple_frame_rate_flag**. For streams where the frame rate is fixed, it may be signaled in the **frame_rate_code** as one of 23.976, 24.0, 25.0, 29.97, 30.0, 50.0, 59.94, or 60.0 Hz. Twenty five hertz is commonly used for European standard-definition television, with 29.97 and 30.0 Hz used for U.S. standard-definition television. Twenty four hertz may be used for movies shot on film. In 25-Hz television systems, such movies have usually been sped up to a frame rate of 25 Hz for broadcast.

video_stream_descriptor	
multiple_frame_rate_flag	0
frame_rate_code	29.97 Hz
MPEG_1_only_flag	0
constrained_parameter_flag	0
still_picture_flag	0
if (MPEG_1_only_flag == 0)	
profile_and_level_indication	Main profile at main level
chroma_format	4:2:0
frame_rate_extension_flag	0

Figure 11.15 Structure of the MPEG-2 video stream descriptor.

Video sequences that are compliant with MPEG-1 are indicated by setting the **MPEG_1_only_flag**, with the **constrained_parameter_flag** set to 1 for MPEG-1 sequences that comply with the MPEG-1 constrained parameters. Both of these flags are set to 0 for MPEG-2 video sequences. The **still_picture_flag** is set only in those sequences that consist only of still pictures. For MPEG-2 video sequences, the profile and level indication (following the format of MPEG-2 video discussed in Chapter 6) and chroma format (4:2:0, 4:2:2, or 4:4:4) are signaled. The use of nonzero values in the frame rate extension fields in the video sequence header (see Chapter 6) is indicated by setting the **frame_rate_extension_flag**.

EXAMPLE 11.5—Video Stream Descriptor

Using the simplified representation, show the contents of a video stream descriptor for an MPEG-2, main profile at main level, video sequence with a frame rate of 29.97 Hz, which includes pictures other than still pictures. The chroma format is 4:2:0.

The video stream descriptor carrying this data is shown in Figure 11.16.

video_stream_descriptor	
multiple_frame_rate_flag	0
frame_rate_code	29.97 Hz
MPEG_1_only_flag	0
constrained_parameter_flag	1
still_picture_flag	0
if (MPEG_1_only_flag == 0)	
profile_and_level_indication	Main profile at main level
chroma_format	4:2:0
frame_rate_extension_flag	0

Figure 11.16 Example of MPEG-2 video stream descriptor. ■

11.6.1.4. MPEG-2 Hierarchy Descriptor

The MPEG-2 hierarchy descriptor (Fig. 11.17) carries information on scalable hierarchies in video, audio, and private data elementary streams. For MPEG-2 video, the **hierarchy_type** may be used to identify an elementary stream as the base layer or an enhancement layer for spatial, SNR or temporal scalability, or data partitioning. An MPEG-2 audio elementary stream may be indicated as the base layer or an MPEG-2 audio extension bit stream. The unique index of a particular elementary stream in a scalable hierarchy is indicated by the **hierarchy_layer_index**. The index of the layer in the hierarchy immediately below the current layer is specified by

hierarchy_descriptor	
hierarchy_type	{MPEG-2 video spatial/SNR/temporal/ data partitioning, MPEG-2 audio extension
hierarchy_layer_index	Unsigned integer
hierarchy_embedded_layer_index	Unsigned integer
hierarchy_channel	Unsigned integer

Figure 11.17 Structure of the MPEG-2 hierarchy descriptor.

```

registration_descriptor
  format_identifier          Unsigned integer
  << additional identification info >>

```

Figure 11.18 Structure of the MPEG-2 registration descriptor.

the **hierarchy_embedded_layer_index**, which has no meaning for a base layer. The **hierarchy_channel** is used to indicate an ordering of channels in a hierarchy, with the most robust of these channels having the lowest value.

Because scalable hierarchies are not supported by either ATSC or DVB, we do not consider the use of this descriptor further.

11.6.1.5. MPEG-2 Registration Descriptor

MPEG-2 provides several opportunities for private data to be included in an MPEG-2 systems bit stream, a feature that is exploited by both DVB and ATSC. The registration descriptor (Fig. 11.18) signals the type of any private data in the bit stream using the **format_identifier**, which is a unique number issued by a registration authority.

11.6.1.6. MPEG-2 Data Stream Alignment Descriptor

The MPEG-2 data stream alignment descriptor, illustrated in Figure 11.19, indicates the type of alignment between the associated video elementary stream and the PES sublayer. The specified alignment applies to those PES packets in which the **data_alignment_indicator** is set to 1. The allowed values of **alignment_type** are “slice or picture,” indicating alignment at the slice or picture layer, “picture,” indicating alignment at the picture layer, “GOP or SEQ,” indicating alignment at the GOP or sequence layer, and “SEQ,” indicating alignment at the sequence layer. Specifying alignment at the slice layer is useful for error robustness because it allows a decoder to resynchronize and begin decoding at the first slice header after an error has occurred.

```

data_stream_alignment_descriptor
  alignment_type          {slice or picture, picture, GOP or SEQ, SEQ}

```

Figure 11.19 Structure of the MPEG-2 data stream alignment descriptor.

EXAMPLE 11.6—Data Stream Alignment Indicator

*Using the simplified representation, show the contents of a data stream alignment descriptor that indicates that each PES packet with **data_alignment_indicator** set to 1 begins with a picture header.*

This descriptor is shown in Figure 11.20.

```

data_stream_alignment_descriptor
  alignment_type          picture

```

Figure 11.20 Example of data stream alignment descriptor. ■

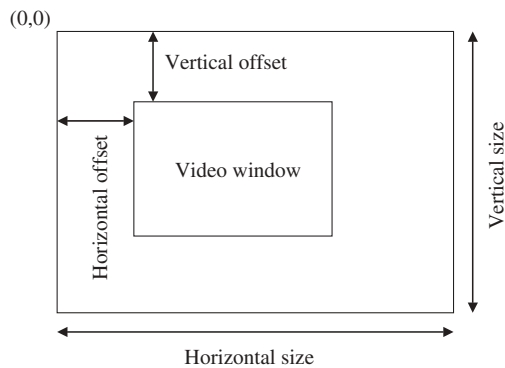


Figure 11.21 Video window filling only part of a display.

11.6.1.7. Descriptors for Video Windows

An MPEG-2 video elementary stream may carry a video sequence that is designed to fill only part of the available display window as illustrated in Figure 11.21. This may arise where a mosaic of video windows, with each window coded in its own elementary stream, is to be displayed. MPEG-2 provides two descriptors to convey information on the window occupied by a video sequence: the target background grid descriptor, which describes the background grid, and the video window descriptor, which specifies the offset in the background grid for one video sequence.

MPEG-2 Target Background Grid Descriptor The MPEG-2 target background grid descriptor specifies a background grid for the associated video elementary stream as shown in Figure 11.22. It specifies the number of pixels vertically and horizontally that make up the grid, which is intended to cover the whole display. The aspect ratio for the grid is also specified, either for the display as a whole or for the individual pixels making up the display. The values of the **horizontal_size** and **vertical_size** fields are calculated as shown in Figure 11.21.

EXAMPLE 11.7—Target Background Grid Descriptor

Using the simplified representation, show the contents of a target background grid descriptor whose target background grid has 720 pixels horizontally and 576 pixels vertically, with a display aspect ratio of 3:4.

```
target_background_grid_descriptor
horizontal_size      Unsigned integer
vertical_size       Unsigned integer
aspect_ratio_information { 1:1, 3:4, 9:16, 1:2.21 }
```

Figure 11.22 Structure of the MPEG-2 target background grid descriptor.

This descriptor is shown in Figure 11.23.

target_background_grid_descriptor	
horizontal_size	720
vertical_size	576
aspect_ratio_information	3:4

Figure 11.23 Example of MPEG-2 target background grid descriptor. ■

video_window_descriptor	
horizontal_offset	Unsigned integer
vertical_offset	Unsigned integer
window_priority	Unsigned integer

Figure 11.24 MPEG-2 video window descriptor.

MPEG-2 Video Window Descriptor The MPEG-2 video window descriptor (Fig. 11.24) specifies the offset (horizontal and vertical) in the grid specified by the target background grid descriptor for an associated video elementary stream as well as a priority value. The available priority values are between 0 and 7, and higher priority windows should be displayed on top of lower priority windows. The values of the **horizontal_offset** and **vertical_offset** fields are calculated as shown in Figure 11.21.

EXAMPLE 11.8—Video Window Descriptor

Using the simplified representation, show a video window descriptor for which a horizontal offset of 134 pixels from the left side of the display and a vertical offset of 0 pixel from the top of the display are specified. This particular window has priority seven, which indicates that it should appear in front of windows with priority between zero and six but behind windows with priority eight or higher.

The descriptor is shown in Figure 11.25.

video_window_descriptor	
horizontal_offset	134
vertical_offset	0
window_priority	7

Figure 11.25 Example of MPEG-2 video window descriptor. ■

11.6.1.8. MPEG-2 CA Descriptor

The MPEG-2 CA descriptor carries a registered identifier for the conditional access system associated with a program or elementary stream using the structure shown in Figure 11.26. The registered identifier for the conditional access system is carried in the **CA_system_ID**, whereas the PID of the transport stream packets containing data associated with the CA system is specified by **CA_PID**. Additional

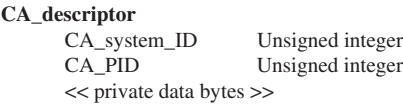


Figure 11.26 Structure of the MPEG-2 CA descriptor.

private data associated with the particular conditional access system may be carried in this descriptor. Any decoder can skip over this private data by using the **descriptor_length** field.

EXAMPLE 11.9—CA Descriptor

Using the simplified representation, show an MPEG-2 CA descriptor indicating that the assigned identifier for the CA used in the associated elementary stream or program is 0x0234 and that information associated with this CA is carried in transport stream packets with PID 0x0934. The remaining data in the CA descriptor is private data with value 0x238F2426.

The descriptor is shown in Figure 11.27.

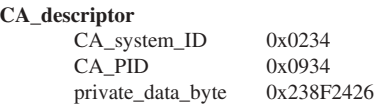


Figure 11.27 Example of MPEG-2 CA descriptor. ■

11.6.1.9. MPEG-2 ISO 639 Language Descriptor

The languages associated with an elementary stream are specified by the ISO 639 language descriptor (see Fig. 11.28), using the format of ISO 639 [4]. This descriptor can also indicate other properties that the audio may possess: “clean effects,” which means that the audio contains no language; “hearing impaired,” which indicates a stream specially designed for the hearing impaired that emphasizes speech over background sounds and effects; and “visual impaired commentary,” which is an audio stream containing additional commentary to assist the visually impaired.

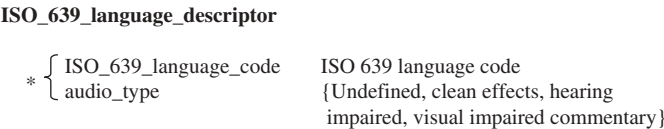


Figure 11.28 Structure of the MPEG-2 ISO 639 language descriptor.

EXAMPLE 11.10—ISO 639 Language Descriptor

Using the simplified representation, show an ISO 639 language descriptor for an English audio service with no special properties.

The descriptor is shown in Figure 11.29.

ISO_639_language_descriptor

ISO_639_language_code	English
audio_type	Undefined

Figure 11.29 Example of MPEG-2 ISO 639 language descriptor. ■

11.6.1.10. MPEG-2 System Clock Descriptor

The purpose of the MPEG-2 system clock descriptor is to specify the accuracy of the system time clock generated by the encoder as shown in Figure 11.30. The accuracy is specified in parts per million (PPM) using the **clock_accuracy_integer** and **clock_accuracy_exponent** fields and is calculated from $clock_accuracy_integer \times 10^{-clock_accuracy_exponent}$ ppm. Where an external source that may be available to a decoder is used by an encoder to generate the system clock, the **external_clock_reference_indicator** may be set to 1.

system_clock_descriptor

external_clock_reference_indicator	flag
clock_accuracy_integer	Unsigned integer
clock_accuracy_exponent	Unsigned integer

Figure 11.30 Structure of the MPEG-2 system clock descriptor.

EXAMPLE 11.11—System Clock Descriptor

Using the simplified representation, show a system clock descriptor describing a system clock based on an external time reference that may be available to the decoder and with an accuracy of 5.4 ppm.

The descriptor is shown in Figure 11.31.

system_clock_descriptor

external_clock_reference_indicator	1
clock_accuracy_integer	540
clock_accuracy_exponent	2

Figure 11.31 Example of MPEG-2 system clock descriptor. ■

```

multiplex_buffer_utilization_descriptor
    bound_valid_flag          flag
    LTW_offset_lower_bound    Unsigned integer
    LTW_offset_upper_bound    Unsigned integer

```

Figure 11.32 Structure of the MPEG-2 multiplex buffer utilization descriptor.

11.6.1.11. MPEG-2 Multiplex Buffer Utilization Descriptor

The MPEG-2 multiplex buffer descriptor provides upper and lower bounds on the occupancy of the multiplex buffer in the system target decoder using the structure shown in Figure 11.32. This information is intended for use by a remultiplexer. The **bound_valid_flag** is used to signal that the other information in the descriptor is valid. The remaining two fields in the descriptor specify the upper and lower bounds, each specified as a number of 90-kHz clock cycles, that is, a multiple of 11.1 μ s.

EXAMPLE 11.12—Multiplex Buffer Utilization Descriptor

Using the simplified representation, show a multiplex buffer utilization descriptor containing valid bounds with a lower bound of 0.389ms and an upper bound of 193.244 ms.

The descriptor is shown in Figure 11.33.

```

multiplex_buffer_utilization_descriptor
    bound_valid_flag          1
    LTW_offset_lower_bound    0x0023
    LTW_offset_upper_bound    0x43F0

```

Figure 11.33 Example of MPEG-2 multiplex buffer utilization descriptor. ■

11.6.1.12. MPEG-2 Copyright Descriptor

The MPEG-2 copyright descriptor (Fig. 11.34) is used to enable identification of rights associated with audio–visual material. The value of the **copyright_identifier** is assigned by a registration authority. A particular value may indicate that rights are owned by a particular organization. Additional information may also be attached, but the format of this information is not standardized by MPEG-2.

11.6.1.13. MPEG-2 Maximum Bit-Rate Descriptor

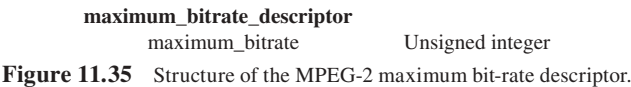
The MPEG-2 maximum bit-rate descriptor, illustrated in Figure 11.35, tells a decoder the maximum bitrate (including transport overhead) that is associated with

```

copyright_descriptor
    copyright_identifier        Unsigned integer
    << additional_copyright_info >>

```

Figure 11.34 Structure of the MPEG-2 copyright descriptor.

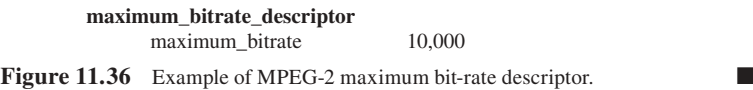


a program or elementary stream in multiples of 400 bit/s (50 byte/s). It is carried in the program map table.

EXAMPLE 11.13—Maximum Bit-Rate Descriptor

Using the simplified representation, show a maximum bit-rate descriptor specifying a maximum bitrate of 4 Mbit/s (500 kbyte/s).

The descriptor is shown in Figure 11.36.

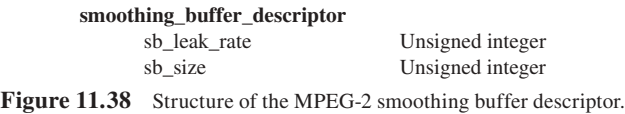
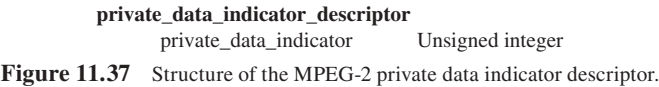


11.6.1.14. MPEG-2 Private Data Indicator Descriptor

The MPEG-2 private data indicator descriptor (Fig. 11.37) carries one field to provide information about private data. The meaning of this field is not defined by MPEG-2.

11.6.1.15. MPEG-2 Smoothing Buffer Descriptor

The MPEG-2 smoothing buffer descriptor provides information on the size and leak rate of the smoothing buffer of the associated elementary stream. Its structure is shown in Figure 11.38. The leak rate is specified in units of 400 bit/s. The buffer size is specified in units of 1 byte.



EXAMPLE 11.14—Smoothing Buffer Descriptor

Using the simplified representation, show an MPEG-2 smoothing buffer descriptor, specifying a leaking rate of 4 Mbit/s and a buffer size of 20,000 bytes.

The descriptor is shown in Figure 11.39.

smoothing_buffer_descriptor	
sb_leak_rate	10,000
sb_size	20,000

Figure 11.39 Example of MPEG-2 smoothing buffer descriptor. ■

STD_descriptor	
leak_valid_flag	flag

Figure 11.40 Structure of the MPEG-2 STD descriptor.

11.6.1.16. MPEG-2 STD Descriptor

The MPEG-2 (system target decoder) STD descriptor (Fig. 11.40) indicates whether the leak method (**leak_valid_flag** = 1) or vbv_delay method (**leak_valid_flag** = 0) is used in the system target decoder.

11.6.1.17. MPEG-2 IBP Descriptor

The MPEG-2 IBP descriptor, illustrated in Figure 11.41, provides information on the GOP structure in an MPEG-2 video elementary stream. The **closed_gop_flag** is set to 1 for elementary streams in which a GOP header immediately precedes each I-picture. Elementary streams in which all GOPs have the same structure (i.e., the same sequence of B- and P-pictures between I-pictures) have the **identical_gop_flag** set to 1. The maximum number of frames that may be transmitted in one GOP is signaled in the **max_gop_length**.

IBP_descriptor	
closed_gop_flag	flag
identical_gop_flag	flag
max_gop-length	Unsigned integer

Figure 11.41 Structure of the MPEG-2 IBP descriptor.

EXAMPLE 11.15—IBP Descriptor

Using the simplified syntax, show an MPEG-2 IBP descriptor, in which a GOP header does not always appear before an I-picture, all GOPs are of the same structure, and the maximum GOP length is 15 pictures.

The descriptor is shown in Figure 11.42.

IBP_descriptor	
closed_gop_flag	0
identical_gop_flag	1
max_gop-length	15

Figure 11.42 Example of MPEG-2 IBP descriptor. ■

11.6.2. MPEG-2 Tables

The MPEG-2 descriptors described in Section 11.6.1 above are carried within one of the three PSI tables (program association table, program map table, or conditional access table) whose syntax is defined by MPEG-2. In this section, the generic syntax of these tables is presented, followed by a discussion of the structure of each table.

11.6.2.1. Generic Table Syntax

Each table is transmitted as one or more sections. Each section may be up to 1024 bytes in length. The generic syntax of the MPEG-2 PSI section is shown in Table 11.9.

The type of table, that is, program association table, program map table, conditional access table, or network information table is specified by the **table_id** field. The values 0x00, 0x01, and 0x02 are assigned to the program association table, conditional access table, and program map table, respectively. Values between 0x40 and 0xFE are available for tables defined outside the scope of MPEG-2 (including the network information table). Other values are not allowed. The identification of table type is followed by the 1-bit **section_syntax_indicator** field whose value is always 1. The reserved bits always take the value 00 and are reserved for future use by the ISO.

The **section_length** field specifies the number of bytes in the current table section following the **section_length** field (i.e., three less than the total number of bytes in the section). This field allows a decoder to identify the location of the end of the section, even if it does not understand the internal syntax of a particular table. For example, an MPEG-2 systems decoder is able to skip over the non-MPEG-2 tables defined by DVB without being able to decode the internal contents of these tables.

Table 11.9 Generic syntax of MPEG-2 PSI tables.

Syntax	Number of bits	Mnemonic
TS_table_section() {		
table_id	8	uimsbf
section_syntax_indicator	1	bslbf
“0”	1	bslbf
reserved	2	bslbf
section_length	12	uimsbf
—		
reserved	2	bslbf
version_number	5	uimsbf
current_next_indicator	1	bslbf
section_number	8	uimsbf
last_section_number	8	uimsbf
—		
CRC_32	32	rpchof

MPEG-2 allows tables longer than 1024 bytes to be transmitted as multiple sections, although many encoders and decoders do not support multisection tables. The various sections of these multisection tables are distinguished by their different values in the **section_number** field.

Each time the contents of a table are changed, the **version_number** field of the table is increased by 1. This allows the new and old contents to be distinguished from one another. An updated table may be transmitted before the new information is current. If this transmission occurs a number of times, it provides a powerful protection against transmission errors introduced in the channel. The **current_next_indicator** is used to indicate whether the table contents being transmitted refer to the current value or the next future value.

The **last_section_number** field specifies the largest value of the **section_number** field used in the table. This enables a decoder to determine the total number of sections in a table when any one of those sections is received.

Each section of each table includes a 32-bit cyclic redundancy code (CRC) (**CRC_32**) so that a decoder can determine confidently whether or not it has been correctly received. This CRC is used on PSI tables but not for other data transmitted in MPEG-2 bit streams because of the catastrophic effect on the quality of the decoded programs of using a corrupted table. Errors introduced into other parts of the bit stream will have effects that are neither as longlasting nor as catastrophic. The mnemonic “rpchhof” indicates a field whose value can be used to detect transmission errors.

11.6.2.2. Program Association Table

When it begins decoding an MPEG-2 systems bit stream, the decoder first searches for the program association table, which can be readily identified because it is always transmitted with a PID of 0x00. The purpose of the program association table is to associate a program number with the PID of the program map table that contains the details of the elementary streams making up that program. The program association table also specifies the PID that is used to transmit the network information table. The program association table must be present in every transport stream and is not scrambled.

The structure of the program map table, shown in Figure 11.43, consists of two parts. The first (**transport_stream_id**) assigns a unique label that can be used to

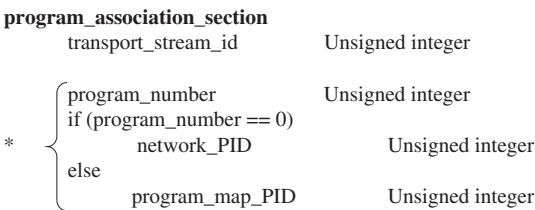


Figure 11.43 Structure of the program association table.

identify this transport stream. The second is a list of one or more programs, for each of which the PID of the transport stream packets carrying the program map table is specified. Program number 0 is used to indicate that the PID used to carry the network information table follows. The program association table carries no descriptors.

A decoder that is able to switch between a number of systems bit streams may monitor all of these bit streams for updates in the program association table to reduce the time required to switch between streams. Without this, there may be long delays associated with a change of systems bit stream.

EXAMPLE 11.16—Program Association Table

Using the simplified representation, show a program association section for a transport_stream_id of 0x1200. The section is to signal that a network information table is carried with PID 0x0010 and that programs 1–3 are carried with PIDs 0x0103, 0x0538, and 0x0456, respectively.

A program association section meeting this specification is shown in Figure 11.44. The order of the programs is arbitrary: In this case we have chosen the sequence 0, 1, 3, 2.

program_association_section	
transport_stream_id	0x1200
program_number	0
network_PID	0x0010
program_number	1
program_map_PID	0x0103
program_number	3
program_map_PID	0x0456
program_number	2
program_map_PID	0x0538

Figure 11.44 Example of program association section. ■

11.6.2.3. Program Map Table

Each section of the program map table provides information about one program. The PID used to carry the program map table section for each program is specified in the program association table. The program map table specifies the PIDs for the elementary streams that make up a program and the PID containing the clock reference for the program. An encoder may assign a separate PID that is used only to carry the clock reference, or the clock reference PID may be shared with one of the elementary stream PIDs. For each elementary stream making up the program, the type of the elementary stream is specified. Normally, the elementary streams of this program are carried in the same transport stream as the section of the program map

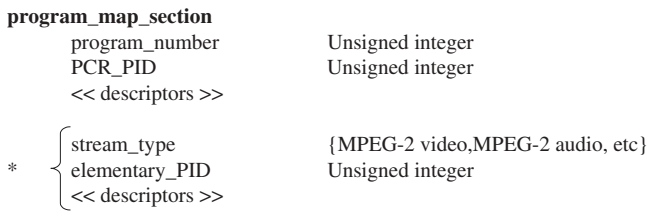


Figure 11.45 Structure of the program map table.

table referring to the program. Program-map-table data must be transmitted for each program carried in a transport stream. The program map table is not scrambled.

The basic structure of the program map table is illustrated in Figure 11.45. The main part of the table contains the program number, which is used by both DVB and ATSC as a means of identifying the group of elementary streams making up the program, and the PID of the transport stream packets carrying the clock reference for this program (**PCR_PID**). Descriptors carried immediately following the **PCR_PID** field refer to the whole program.

For each elementary stream making up the program, the program map table carries the stream type and the PID of the transport stream packets carrying the elementary stream. The stream type specifies the elementary stream decoder to which arriving packets are sent at the decoder. Descriptors following these fields refer only to one elementary stream. For video elementary streams, the video stream descriptor identifies basic coding parameters of the elementary stream, including frame rate, profile and level, and chroma format. For MPEG-2 audio elementary streams, the audio stream descriptor identifies the audio coder used including layer information.

EXAMPLE 11.17—Program Map Table

Using the simplified representation, show a program map table containing one video and one audio elementary stream. The PCR is carried in transport stream packets with the same PID as the video and the maximum bit rate for the program, including all elementary streams and systems overhead, is 5 Mbit/s.

The program map table is shown in Figure 11.46.

The video elementary stream is carried in transport stream packets with PID 0x0308 and is described by two descriptors: the video stream descriptor and data_stream_alignment descriptor. The video stream descriptor shows that the frame rate of the video is 25 Hz, that it is MPEG-2 main profile at main level, and that the chroma format is 4:2:0. The data stream alignment descriptor tells a decoder that the elementary stream is aligned to the PES layer at the slice layer.

The audio elementary stream is MPEG-2, Layer-2 audio. The ISO 639 language descriptor shows that the language of the audio stream is English. The value “undefined” for **audio_type** means that the audio stream does not have special properties such as being designed for hearing impaired.

program_map_section	
program_number	2
PCR_PID	0x0308
maximum_bitrate_descriptor	
maximum_bitrate	5 Mbit/s
stream_type	MPEG-2 video
elementary_PID	0x0308
video_stream_descriptor	
multiple_frame_rate_flag	0
frame_rate_code	25 Hz
MPEG_1_only_flag	0
constrained_parameter_flag	0
still_picture_flag	0
profile_and_level_indication	Main profile at main level
chroma_format	4:2:0
frame_rate_extension_flag	0
data_stream_alignment_descriptor	
alignment_type	Slice layer
stream_type	MPEG-2 BC audio
elementary_PID	0x0309
audio_stream_descriptor	
free_format_flag	0
ID	1
layer	2
variable_rate_audio_indicator	0
ISO_639_language_descriptor	
ISO_639_language_code	English
audio_type	Undefined

Figure 11.46 Example of program map table. ■

11.6.2.4. Conditional Access Table

The conditional access table carries information on conditional access schemes that may be used to control access to programs by scrambling. It is only transmitted where conditional access is used. If present, the conditional access table is transmitted with PID 0x01. Its major purposes are to flag the use of conditional access, to identify the conditional access system in use, and to specify the locations of conditional access data in a transport stream. The conditional access table may carry descriptors providing system-wide configuration management information for the conditional access system. This might include the delivery of keys to be used to decrypt encrypted programs.

All data in the conditional access table is carried in descriptors as shown in the structure of Figure 11.47.

The CA descriptor is usually the only descriptor carried in the conditional access table. Its contents point to system wide access management or control information, in contrast to the information associated with particular programs that is carried by this descriptor when it appears in the program map table.

conditional_access_section
<< descriptors >>

Figure 11.47 Structure of the conditional access table.

EXAMPLE 11.18—Conditional Access Table

Using the simplified representation, show a conditional access table that carries one CA descriptor that identifies the CA system in use and PID in which information associated with the CA system it carried.

The conditional access section is shown in Figure 11.48.

conditional_access_section	
CA_descriptor	
CA_system_ID	0x0234
CA_PID	0x0934

Figure 11.48 Example of conditional access table. ■

11.6.2.5. Network Information Table

The purpose of the network information table is to carry information on the physical characteristics of the network being used to deliver the transport stream and possibly on other networks being used to deliver other transport streams. Its contents are not defined by MPEG-2. For a satellite delivery network, the types of information that might be carried in the network information table include the transmission frequency, modulation characteristics, and polarization. The network information table is transmitted in its own PID as specified in the program association table.

11.6.3. Overheads Due to PSI

An MPEG-2 decoder cannot extract the elementary streams making up a program from a systems bit stream without first receiving the PSI data describing the structure of the multiplex. Typically, random access points in digital television occur at GOP boundaries. An approximate length of the PSI can be obtained by assuming that the PSI is transmitted once per GOP, that each table occupies exactly one transport stream packet, and that each program map section (describing one program) is transmitted in its own transport stream packet.

EXAMPLE 11.19—PSI Overheads

For typical standard-definition television, random access is usually limited by the GOP structure of the video to approximately every half second (15 frames for 30 frames per second and 12 frames for 25 frames per second). For a systems bit stream at 4 Mbit/s containing one program, the overhead if PSI data is transmitted twice per second is likely to be approximately 6 kbyte/s, which is equivalent to 1.2% of the systems bit-stream capacity.

11.7. MPEG-2 DECODER OPERATION

When beginning to decode a transport stream, an MPEG-2 decoder undertakes three major tasks: synchronization to the transport stream, acquisition of PSI data, and program selection and decoding. Each task must be completed before the following task can begin.

11.7.1. Synchronization to Transport Stream

The sequence of operations required to begin decoding a program contained in a transport stream is as follows. First, the decoder must acquire synchronization with the transport stream, identifying the locations of the boundaries of the transport stream packets. Because there is no guarantee that the **sync_byte** value (0x47) cannot appear elsewhere in a transport stream, this synchronization process involves testing the **sync_byte** value for several successive packets. For some delivery systems, such as an ATM network, the underlying transport layer may assist the decoder in transport stream synchronization.

11.7.2. PSI Decoding

Following synchronization, the decoder scans the transport stream to find the program association table, which always uses a PID of 0x00. From the program association table, the decoder obtains one or more PIDs that are used to carry the program map table, which contains lists of PIDs of elementary streams making up the programs contained in the transport stream. The viewer is now able to select a program for decoding, after which the systems decoder extracts the elementary stream data for that program and discards all other data in the transport stream. Each of the elementary streams making up the program is passed to the appropriate elementary stream decoder. After decoding, the contents of each elementary stream are displayed.

Random access within transport streams is required where a decoder is able to switch between two or more streams. This may happen in terrestrial broadcast when changing frequencies. It may also occur when a decoder connected to two or more types of distribution network (e.g., cable and satellite) changes from one network to another. Random access can be facilitated by the regular retransmission of tables within each transport stream. MPEG-2 does not specify how often tables should be retransmitted. This approach is used in DVB. Random access may also be facilitated by placing additional restrictions on the transport stream. For example, the PIDs of the elementary streams making up a program may be required to be contiguous. This approach is used in ATSC.

11.7.3. Program Reassembly

An MPEG-2 decoder is only required to be capable of decoding one program, even though a transport stream may contain elementary streams for many programs.

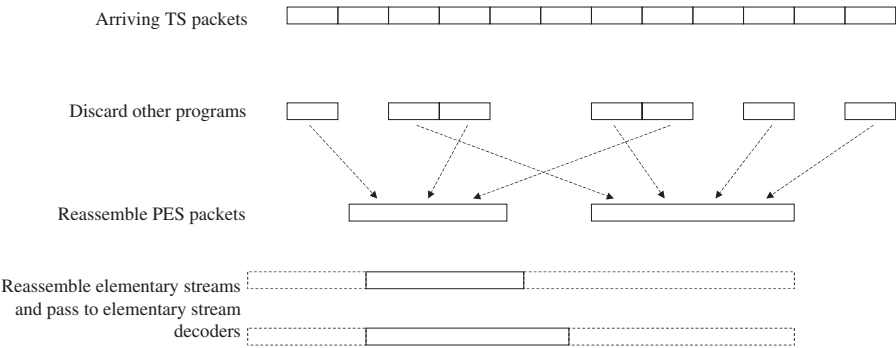


Figure 11.49 Program reassembly in MPEG-2 decoder.

The transport stream packets for a particular elementary stream are identified by their unique PID. Program reassembly in a decoder begins with breaking up of the MPEG-2 systems bit stream into transport stream packets. The decoder scans the PIDs in the transport stream packet headers and discards all those transport stream packets whose PID is not associated with an elementary stream associated with the current program. PES packets are then reassembled for each elementary stream that forms part of the current program, from which the elementary streams are extracted and passed to an elementary stream decoder. This process is illustrated in Figure 11.49.

EXAMPLE 11.20—MPEG-2 Transport Stream Decoding

An example of five packets from an MPEG-2 transport stream is shown in Table 11.10. Each table entry shows the contents of 1 byte of the transport stream. The first 16 bytes are shown in the top row, the next 16 on the second row, and so on. For this example stream, identify the contents of the header of each transport stream packet.

In this example, the first byte has value 0x47 and may be a transport packet **sync_byte**. This can be confirmed by observing that the value 188 byte later (0x0B, 0xC) has the same value as do all the entries at higher multiples of 188 bytes. In this example, all sync bytes are shown in *italics*.

The PID of the first transport stream packet is 0x0233. Unless the decoder has already read the program association table and program map table for this transport stream, this packet cannot be decoded. The 13-bit PID of the second transport stream packet is 0x0000, indicating the presence of the program association table. The **payload_unit_start_indicator** is also set to 1, showing that a new program association section starts in this transport stream packet. The first payload byte of the transport packet (at location (0x0C,0)) has value 5, indicating an offset of 5 bytes to the start of the program association section (at location (0x0C,6)) where a value of zero corresponds to **table_id** = 0, which is consistent with the program association section.

Table 11.10 Example of MPEG-2 transport stream.

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00	47	02	33	15	E4	C3	74	04	D2	71	9D	CA	EB	BC	2D	67
01	EF	EA	69	E4	0E	5A	D0	02	23	33	32	9A	45	32	03	BF
02	71	EE	77	6B	D8	86	33	AC	D6	05	AE	61	D4	80	B5	6D
03	4D	30	31	AE	4D	8A	26	B2	60	DC	DA	97	7F	E6	D2	A5
04	D1	A9	57	4A	57	88	BA	4F	D6	91	5E	B3	8B	71	B1	9F
05	CB	F4	85	E1	2C	FA	45	40	E0	BC	22	03	E4	32	4C	A9
06	48	78	10	FD	95	6C	83	55	6E	39	94	C2	87	A3	35	61
07	C8	AE	76	91	CB	0F	9A	0C	6A	4E	DF	03	C4	F8	FD	C9
08	70	7F	36	A4	51	F5	BA	69	BE	44	70	EE	AE	36	D6	A0
09	22	35	9B	A1	5E	93	73	0B	06	50	03	62	AE	17	09	9C
0A	9B	04	04	30	96	0E	5E	A1	B7	B1	15	74	71	5A	27	AC
0B	B2	BA	7A	8E	1E	73	B7	E4	45	41	DD	3B	47	40	00	15
0C	05	14	A4	30	D8	2C	00	00	70	57	50	5D	64	97	1E	09
0D	75	DE	EF	43	29	DF	3C	A5	F7	AA	DE	02	23	D1	6E	E3
0E	BC	AF	58	2A	27	30	6C	DB	7D	D0	75	75	73	69	E6	01
0F	4C	0C	B1	A6	FB	8D	66	32	A0	BB	60	02	6B	C0	CB	EB
10	D8	5E	9E	BB	31	E7	91	A1	3C	8C	EE	55	A7	64	A0	B2
11	65	69	A7	D6	5F	6C	98	90	B7	82	C6	7D	2F	B3	FB	CE
12	B4	7C	1D	AA	5D	23	91	D2	AC	FF	F6	0F	5C	8C	43	98
13	0C	92	B3	F6	C0	BD	6E	A2	CD	15	F2	EA	9A	40	DF	83
14	BB	6C	F6	12	8D	4A	DB	55	AE	0D	5B	7F	6F	8F	9D	1D
15	E5	C1	CA	D0	AB	33	45	A0	89	0F	16	45	68	79	E8	98
16	54	7A	98	29	D4	F4	98	07	CF	9C	B3	17	6C	60	2A	D5
17	D6	73	F4	25	DE	C4	71	9E	47	02	33	16	30	7D	68	76
18	9C	12	50	9B	2C	9E	3E	96	81	76	8A	F1	57	66	4E	69
19	49	64	80	B8	4E	1C	71	77	03	A9	B9	48	43	B5	C8	FC
1A	79	E7	73	CD	D4	2A	64	85	B7	91	75	71	16	71	5D	4D
1B	DA	C2	F3	8E	03	98	D0	FA	38	B4	85	EE	B6	3A	73	2C
1C	F8	5B	0C	C1	E5	49	40	EE	21	F0	B3	D9	35	74	14	D9

(continued)

Table 11.10 (Continued)

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
1D	8F	51	5F	DE	5F	12	33	0C	91	1F	85	1D	C5	60	D2	0B
1E	99	F2	49	E3	1A	10	3B	EE	10	43	FF	36	7F	4A	AC	F5
1F	C4	AA	21	18	03	49	D1	FC	04	D1	9F	8F	3E	D2	43	C0
20	A8	36	9A	9A	A8	2E	A2	2B	8A	9F	AF	AD	E0	03	4F	C7
21	4E	ED	AD	13	12	03	3A	84	75	B4	95	82	13	31	61	46
22	C5	50	A3	FC	80	F2	D3	EA	1C	CF	E8	28	1F	C3	B8	A6
23	C1	A9	E2	45	47	00	00	16	D9	57	77	E9	3A	DC	A8	E4
24	7C	FE	5F	88	2E	80	6C	A9	AC	F5	31	1C	90	F8	06	DE
25	06	84	31	B7	40	EF	23	85	E5	F1	55	6F	78	26	22	88
26	B9	66	5B	49	DE	A0	3D	FA	A3	3A	AE	AA	22	05	43	1D
27	11	DA	2E	08	BB	89	46	5E	03	E3	DD	41	91	28	98	54
28	A8	DD	91	FB	CA	27	D5	31	A3	AB	C5	61	71	7B	9B	2D
29	00	CA	83	36	1A	28	68	68	0D	F1	26	62	4F	2B	E5	52
2A	BB	69	66	81	2B	86	A4	04	D6	CD	B2	76	15	D2	31	72
2B	03	4F	E0	D5	55	E1	7A	8F	9D	A9	9D	AF	82	B6	83	9B
2C	F7	D2	51	96	21	41	CD	AA	03	8F	74	E7	48	10	7A	FB
2D	EC	8F	A6	C5	1B	00	8A	01	73	32	C9	9E	03	E4	C2	E8
2E	C2	61	54	81	90	C4	C7	7B	CD	78	33	94	AA	AD	F1	C5
2F	47	02	33	17	A1	CA	72	86	2B	21	38	1B	24	74	C9	47
30	39	E8	01	96	8A	A7	50	3B	6A	4C	AC	F0	57	90	1E	2B
31	47	8E	7C	F3	3B	7A	86	CA	31	E8	EC	03	C4	F2	D0	EC
32	32	AC	ED	58	98	9D	00	FB	E6	B1	70	B3	9C	4C	DB	1C
33	4A	18	65	55	F1	D6	42	0A	01	93	BE	CE	A3	40	24	A6
34	F2	D0	EE	4F	44	89	29	36	37	A6	0D	3A	AA	4F	4E	B8
35	F4	21	11	20	2A	E9	22	9D	44	38	B6	8C	F0	54	B4	F1
36	94	E1	BF	61	B9	29	F4	32	C6	9D	29	07	49	F8	F3	3A
37	F5	AE	0E	99	64	37	2E	13	01	C9	04	E0	5A	B8	F7	27
38	29	50	07	5B	06	CB	FF	1C	9F	21	4F	22	39	65	22	3D
39	ED	64	82	17	05	28	D8	E1	2F	FD	B6	DF	7A	7E	49	0F
3A	43	2F	EA	1F	03	5E	B2	E3	98	28	51	3B				

The last two packets in Table 11.10 have PID 0x0233, which is the same as the first packet. The **continuity_counter** fields of these three packets can be seen to be increasing by 1. ■

11.8. USE OF MPEG-2 SYSTEMS IN DIGITAL TELEVISION

This section describes the specific use of MPEG-2 systems in the ATSC and DVB digital television standards. Both systems place a number of restrictions on the MPEG-2 systems.

11.8.1. Use of MPEG-2 Systems in ATSC

MPEG-2 systems provides the multiplex for ATSC digital television. The ATSC standard, however, places a number of restrictions on the MPEG-2 implementation of multiplexing and PSI, which are discussed in this section.

11.8.1.1. Implementation of Multiplexing in ATSC

The MPEG-2 transport stream is the only multiplex used; the program stream is not used.

Within the PES header

- scrambling is not used (this means that scrambling in ATSC is based on scrambling of transport stream packets);
- the ESCR is not transmitted;
- the optional CRC is not transmitted;
- private data is not carried; and
- fields relating to the program stream are coded as zero or not carried.

For video elementary streams, the **data_alignment_indicator** in the PES header always has value 1, and the **PES_packet_length** is specified as 0. This means that the length of PES packets is not bounded by the PES header and that alignment is guaranteed between the video and PES layers in every PES packet.

AC-3 audio is assigned a value of 0xBD for the **stream_id** in the PES header.

Still pictures are not supported.

11.8.1.2. Implementation of PSI in ATSC

MPEG-2 PSI is used by ATSC, with extensions to add tables and descriptors of its own, using values of **descriptor_tag** and **table_id** left for private definition by MPEG-2. These extensions are described fully in Chapter 12.

Table 11.11 Example for Program 51 (0x0033).

Name	PID value
base_PID	0x0330
PMT_PID	0x0330
Video_PID	0x0331
PCR_PID	0x0331
Audio_PID	0x0334

ATSC assigns the **stream_type** value 0x81 in the PMT to AC-3 audio.

Programs carried by a transport stream that comply with the requirements of ATSC are indicated by the placement of a registration descriptor in the program map table, with the **format_identifier** set to 0x4741 3934. AC-3 audio streams are carried as private data and indicated by associating a registration descriptor with **format_identifier** set to 0x4143 2D33 with the audio elementary stream in the program map table.

Using the MPEG-2 PSI, random access to a transport stream is limited to those points where a complete set of PSI is transmitted. Rather than limit random access to these points, ATSC defines program numbers and assigns fixed PIDs to the elementary streams making up each program. For each program number (assigned in the program association table) between 1 and 255, a *base_pid* is calculated by multiplying the program number by 16. The PMT for this program is carried in transport stream packets whose PID is $\text{PMT_PID} = \text{base_PID} + 0x0000$. The video elementary stream (*Video_PID*) and PCR (*PCR_PID*) are both carried with PID $\text{base_PID} + 0x0001$. The primary audio for the program is carried in $\text{Audio_PID} = \text{base_PID} + 0x0004$. An example of this calculation is shown in Table 11.11.

11.8.2. Use of MPEG-2 Systems in DVB

MPEG-2 systems provides the multiplex for DVB digital television. The DVB standards, however, place a number of restrictions on the MPEG-2 implementation of multiplexing and PSI, which are discussed in this section.

11.8.2.1. Implementation of Multiplexing in DVB

DVB does not use the following features of MPEG-2 systems [5]:

- the use of trick modes (such as fast forward, fast rewind, pause);
- ESCR;
- private data in PES headers; and
- the program stream, including all associated signaling.

A DVB decoder is required to be able to scan any valid MPEG-2 systems bit stream but may ignore any components associated with unused features.

11.8.3. Implementation of PSI in DVB

MPEG-2 PSI is used by DVB, along with additional tables and descriptors of its own, using values of **descriptor_tag** and **table_id** left for private definition by MPEG-2. These extensions are described fully in Chapter 12.

11.9. CONCLUSION

MPEG-2 systems provide a range of services for multiplexing elementary streams, timing and synchronization, buffer management, and control.

In following chapters, the following are of particular importance:

- The value of the **PID** field in the transport stream packet header is associated with one source of data, either an elementary stream or a PSI table.
- A particular transport stream can be uniquely identified by the **transport_stream_id** in the program association table.
- The type of an elementary stream can be identified from the **stream_id** field of the PES packet header and the **stream_type** field of the program map table.

PROBLEMS

11.1 Fill in the empty boxes in the following table of transport stream overheads in Table 11.12.

Table 11.12 Transport stream overheads for Q 11.1.

Frames per second	Slices per frame	Number of programs	Transport stream rate	Overhead
30	30	5	20 Mbit/s	2%
30	30	1		
30	1	5	20 Mbit/s	

11.2 For the bitstream in Table 11.13:

- (a) Identify the locations at which transport stream packets start.
- (b) For each transport stream packet, find its PID and ensure that all values of the continuity counter are valid.
- (c) Find the packet(s) that contain the program map table and ensure that the correct value of **table_id** is present.

Table 11.13 Example transport stream bit stream for Q 11.2.

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00	43	2F	EA	1F	03	5E	B2	E3	98	28	51	3B	B2	E3	98	47
01	02	33	15	E4	C3	86	74	04	D2	71	9D	CA	EB	BC	2D	67
02	EF	EA	69	E4	0E	5A	D0	02	23	33	32	9A	45	32	03	BF
03	71	EE	77	6B	D8	86	33	AC	D6	05	AE	61	D4	80	B5	6D
04	4D	30	31	AE	4D	8A	26	B2	60	DC	DA	97	7F	E6	D2	A5
05	D1	A9	57	4A	57	88	BA	4F	D6	91	5E	B3	8B	71	B1	9F
06	CB	F4	85	E1	2C	FA	45	40	E0	BC	22	03	E4	32	4C	A9
07	48	78	10	FD	95	6C	83	55	6E	39	94	C2	87	A3	35	61
08	C8	AE	76	91	CB	0F	9A	0C	6A	4E	DF	03	C4	F8	FD	C9
09	70	7F	36	A4	51	F5	BA	69	BE	44	70	EE	AE	36	D6	A0
0A	22	35	9B	A1	5E	93	73	0B	06	50	03	62	AE	17	09	9C
0B	9B	04	04	30	96	0E	5E	A1	B7	B1	15	74	71	5A	27	AC
0C	B2	BA	7A	8E	1E	73	B7	E4	45	41	DD	47	40	00	15	05
0D	14	A4	30	D8	2C	00	3C	00	70	57	50	5D	64	97	1E	09
0E	75	DE	EF	43	29	DF	3C	A5	F7	AA	DE	02	23	D1	6E	E3
0F	BC	AF	58	2A	27	30	6C	DB	7D	D0	75	75	73	69	E6	01
10	4C	0C	B1	A6	FB	8D	66	32	A0	BB	60	02	6B	C0	CB	EB
11	D8	5E	9E	BB	31	E7	91	A1	3C	8C	EE	55	A7	64	A0	B2
12	65	69	A7	D6	5F	6C	98	90	B7	82	C6	7D	2F	B3	FB	CE
13	B4	7C	1D	AA	5D	23	91	D2	AC	FF	F6	0F	5C	8C	43	98
14	0C	92	B3	F6	C0	BD	6E	A2	CD	15	F2	EA	9A	40	DF	83
15	BB	6C	F6	12	8D	4A	DB	55	AE	0D	5B	7F	6F	8F	9D	1D
16	E5	C1	CA	D0	AB	33	45	A0	89	0F	16	45	68	79	E8	98
17	54	7A	98	29	D4	F4	98	07	CF	9C	B3	17	6C	60	2A	D5
18	D6	73	F4	25	DE	C4	71	47	02	33	16	30	7D	68	76	03
19	9C	12	50	9B	2C	9E	3E	96	81	76	8A	F1	57	66	4E	69
1A	49	64	80	B8	4E	1C	71	77	03	A9	B9	48	43	B5	C8	FC
1B	79	E7	73	CD	D4	2A	64	85	B7	91	75	71	16	71	5D	4D
1C	DA	C2	F3	8E	03	98	D0	FA	38	B4	85	EE	B6	3A	73	2C
1D	F8	5B	0C	C1	E5	49	40	EE	21	F0	B3	D9	35	74	14	D9
1E	8F	51	5F	DE	5F	12	33	0C	91	1F	85	1D	C5	60	D2	0B
1F	99	F2	49	E3	1A	10	3B	EE	10	43	FF	36	7F	4A	AC	F5
20	C4	AA	21	18	03	49	D1	FC	04	D1	9F	8F	3E	D2	43	C0
21	A8	36	9A	9A	A8	2E	A2	2B	8A	9F	AF	AD	E0	03	4F	C7
22	4E	ED	AD	13	12	03	3A	84	75	B4	95	82	13	31	61	46
23	C5	50	A3	FC	80	F2	D3	EA	1C	CF	E8	28	1F	C3	B8	A6
24	C1	A9	E2	00	00	16	D9	57	77	E9	3A	3A	3A	DC	A8	E4
25	7C	FE	5F	88	2E	80	6C	A9	AC	F5	31	1C	90	F8	06	DE
26	06	84	31	B7	40	EF	23	85	E5	F1	55	6F	78	26	22	88
27	B9	66	5B	49	DE	A0	3D	FA	A3	3A	AE	AA	22	05	43	1D
28	11	DA	2E	08	BB	89	46	5E	03	E3	DD	41	91	28	98	54
29	A8	DD	91	FB	CA	27	D5	31	A3	AB	C5	61	71	7B	9B	2D
2A	00	CA	83	36	1A	28	68	68	0D	F1	26	62	4F	2B	E5	52
2B	BB	69	66	81	2B	86	A4	04	D6	CD	B2	76	15	D2	31	72

Table 11.13 (Continued)

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
2C	03	4F	E0	D5	55	E1	7A	8F	9D	A9	9D	AF	82	B6	83	9B
2D	F7	D2	51	96	21	41	CD	AA	03	8F	74	E7	48	10	7A	FB
2E	EC	8F	A6	C5	1B	00	8A	01	73	32	C9	9E	03	E4	C2	E8
2F	C2	61	54	81	90	C4	C7	7B	CD	78	33	94	AA	AD	F1	47
30	02	33	17	A1	CA	72	86	2B	21	98	38	1B	24	74	C9	47
31	39	E8	01	96	8A	A7	50	3B	6A	4C	AC	F0	57	90	1E	2B
32	47	8E	7C	F3	3B	7A	86	CA	31	E8	EC	03	C4	F2	D0	EC
33	32	AC	ED	58	98	9D	00	FB	E6	B1	70	B3	9C	4C	DB	1C
34	4A	18	65	55	F1	D6	42	0A	01	93	BE	CE	A3	40	24	A6
35	F2	D0	EE	4F	44	89	29	36	37	A6	0D	3A	AA	4F	4E	B8
36	F4	21	11	20	2A	E9	22	9D	44	38	B6	8C	F0	54	B4	F1
37	94	E1	BF	61	B9	29	F4	32	C6	9D	29	07	49	F8	F3	3A
38	F5	AE	0E	99	64	37	2E	13	01	C9	04	E0	5A	B8	F7	27
39	29	50	07	5B	06	CB	FF	1C	9F	21	4F	22	39	65	22	3D
3A	ED	64	82	17	05	28	D8	E1	2F	FD	B6	DF	7A	7E	49	0F

- 11.3** Construct transport stream headers (including offsets to the start of any PSI sections and start codes associated with PES packet headers) for the following data: **transport_error_indicator** = 0, **payload_unit_start_indicator** = 0, **transport_priority** = 0, **PID** = 563, **ts_scrambling_control** = 0, **ts_adaptation_control** = 1, **continuity_counter** = 6.
- 11.4** Construct transport stream headers (including offsets to the start of any PSI sections and start codes associated with PES packet headers) for the following data: **transport_error_indicator** = 0, **payload_unit_start_indicator** = 1, **transport_priority** = 0, **PID** = 563, **ts_scrambling_control** = 0, **ts_adaptation_control** = 1, **continuity_counter** = 6, for a transport stream packet containing PES packet data.
- 11.5** Construct transport stream headers (including offsets to the start of any PSI sections and start codes associated with PES packet headers) for the following data: **transport_error_indicator** = 0, **payload_unit_start_indicator** = 1, **transport_priority** = 0, **PID** = 63, **ts_scrambling_control** = 0, **ts_adaptation_control** = 1, **continuity_counter** = 6, for a transport stream carrying PSI data with offsets of 34 and 93 bytes to the starts of the PSI sections.
- 11.6** Construct a program association table for a transport stream carrying two programs whose program map tables have **PID** 0x0123 and 0x38F, with **transport_stream_id** = 0x083E. The network information table for this transport stream is carried in packets with **PID** 0x0562.
- 11.7** Construct a program map table for a program consisting of one video and one audio elementary stream, with the **PCR** carried in with the same **PID** as the video elementary stream. The video elementary stream is carried with **PID** 0x0437 and is the main profile at main level with 30-Hz frame rate and 4:2:0 chroma format. The audio elementary stream is constant bit-rate MPEG-2 Layer 2 audio with **free_format_flag** = 0, **ID** = 1 and is carried with **PID** 0x0346.

11.8 Use the data in Table 11.14, which describes the programs and elementary streams carried in an MPEG-2 transport stream, to construct a program association table and program map tables. The transport stream has a maximum bit rate of 20Mbit/s, and **transport_stream_id** is 0x0034. In answering this question, it may be necessary to assign PIDs or other parameters. The network information table is carried in transport stream packets with PID 0x001F.

Table 11.14 Parameters of program and their constituent elementary streams for Q 11.6.

Program number	PID	Elementary stream type	Elementary stream description
1	0x0365	MPEG-2 video	Main profile at main level; 4:2:0; 25 Hz; aligned at slice layer
	0x366	MPEG-2 audio	English language; constant rate; Layer 2; ID = 1; free_format_flag = 0
	0x0365	PCR	Elementary stream carrying PCR for this program
2	0x0385	MPEG-2 video	Main profile at main level; 4:2:0; 30 Hz; aligned at picture layer
	0x386	MPEG-2 audio	English language; constant rate; Layer 2; ID = 1; free_format_flag = 0
	0x387	MPEG-2 audio	French language; constant rate; Layer 2; ID = 1; free_format_flag = 0
	0x0385	PCR	Elementary stream carrying PCR for this program

- 11.9** Calculate the overheads for the transport stream sublayer in the following cases:
- (a) 30 slices per picture, 24 pictures per second, with each slice occupying one PES packet;
 - (b) 36 slices per picture, 25 pictures per second, with each slice occupying two PES packets; and
 - (c) 36 slices per picture, 25 pictures per second, with each slice occupying four PES packets.
- 11.10** Calculate the overheads due to the PES sublayer in the following cases:
- (a) 30 slices per picture, 24 pictures per second, with each slice occupying one PES packet;
 - (b) 36 slices per picture, 25 pictures per second, with each slice occupying two PES packets; and
 - (c) 36 slices per picture, 25 pictures per second, with each slice occupying four PES packets.
- 11.11** Construct an MPEG-2 target background grid descriptor and MPEG-2 video window descriptor for a 720×576 pixel display with aspect ratio 4:3, containing a 352×288 window offset 50 pixels horizontally and 30 pixels vertically from the top left of the display;

- 11.12 Construct an MPEG-2 target background grid descriptor and MPEG-2 video window descriptor for a 720×576 pixel display with aspect ratio 4:3, containing a 352×288 window offset, 50 pixels horizontally and 30 pixels vertically from the bottom right of the display;
- 11.13 Construct an MPEG-2 IBP descriptor for an MPEG-2 video elementary stream in which each I-picture is immediately preceded by a GOP header, and each GOP contains exactly 12 pictures.
- 11.14 Construct an MPEG-2 video stream descriptor for a video elementary stream that carries an MPEG-1 constrained parameters bit stream with a frame rate of 30 Hz.
- 11.15 Construct an MPEG-2 ISO 639 language descriptor describing an audio elementary stream containing English language with no defined type and a French service for the hearing impaired.
- 11.16 Construct an MPEG-2 system clock descriptor for a system clock with accuracy 34 ppm.
- 11.17 Construct an MPEG-2 system clock descriptor for a system clock with accuracy 45 ppm.
- 11.18 Calculate the values of **CR_base** and **CR_ext** for the following values of the system time clock:
- (a) 20,193,325;
 - (b) 5384.
- 11.19 Write down the closest time stamp values to the values of the system time clock in Q 11.18.
- 11.20 For the transport stream packet payload shown in Table 11.15, identify the **table_id** of each section appearing and the contents of each of these sections.

Table 11.15 Transport stream packet payload for Q 11.20.

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00	EF	EA	69	E4	0E	5A	D0	02	23	33	32	9A	45	32	03	BF
01	71	EE	77	6B	D8	86	33	AC	D6	05	AE	61	D4	80	B5	6D
02	2F	00	14	AE	4D	8A	26	B2	60	DC	DA	97	7F	E6	D2	A5
03	D1	A9	57	4A	57	88	BA	FF	FF	FF	FF	FF	FF	FF	FF	FF
04	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
05	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
06	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
07	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
08	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
09	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
0A	FF	FF	FF	FF	FF	FF	FF	FF	FF							

REFERENCES

1. ISO/IEC 13818-1, *Information technology – Generic coding of moving pictures and associated audio information: Systems*, Geneva, Switzerland: International Standards Organisation, 1996.

2. While the structure shown in Figure 11.1 is consistent with the MPEG-2 standards, it is not mandated. There is no necessity, for example, that there exist a discrete entity that performs video encoding and

produces a video bit stream. What is required is that the encoder accept video and audio inputs and produce a valid systems bit stream.

3. As discussed above for the encoder, the MPEG-2 standards do not require that the decoder adopt this structure. What is required is that the decoder be capable of reconstructing video and audio signals from a received systems bit stream. Subject to this constraint, the internal architecture of the decoder may have any structure.
4. ISO 639, Code for the representation of names of languages.
5. Digital Video Broadcasting (DVB); Implementation guidelines for the use of MPEG-2 Systems, Video and Audio in satellite, cable and terrestrial broadcasting applications, ETR 154, Sophia Antipolis: ETSI, 1997.